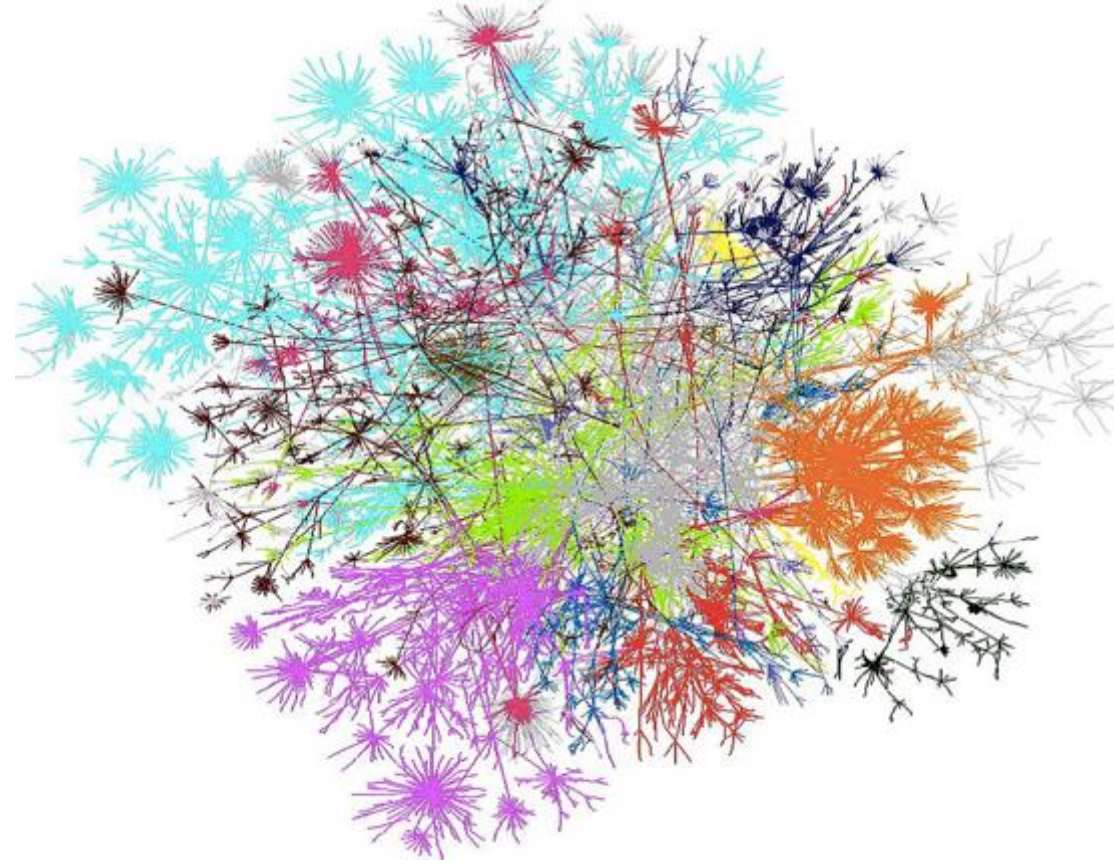


CS 31204: Computer Networks – Internet Routing

Department of Computer Science
and Engineering



INDIAN INSTITUTE OF TECHNOLOGY
KHARAGPUR



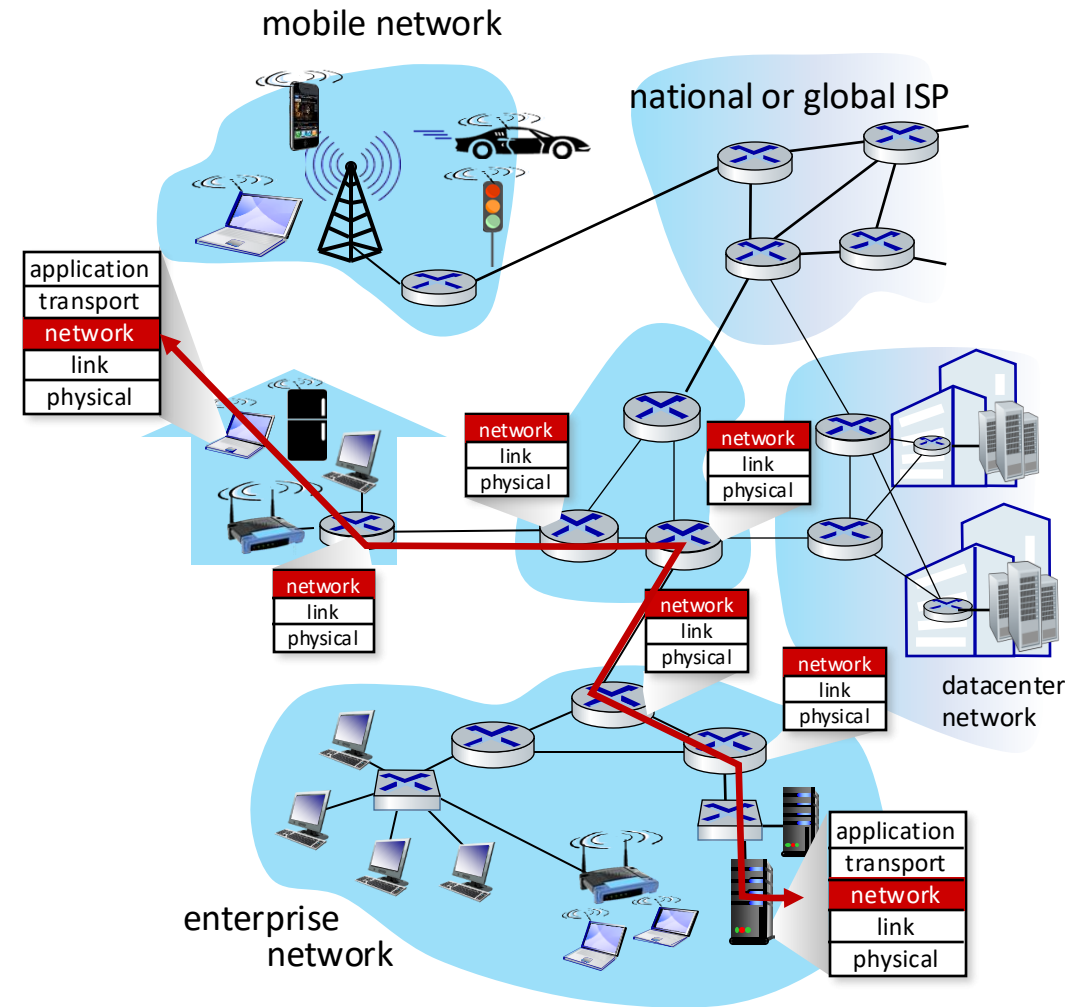
Abhijnan Chakraborty
abhijnan@cse.iitkgp.ac.in

Sandip Chakraborty
sandipc@cse.iitkgp.ac.in

Routing protocols

Routing protocol goal: determine “good” paths (equivalently, routes), from sending hosts to receiving host, through network of routers

- **path:** sequence of routers packets traverse from given initial source host to final destination host
- **“good”:** least “cost”, “fastest”, “least congested”



Making routing scalable

Scale: billions of destinations:

- can't store all destinations in routing tables!
- routing table exchange would swamp links!

Administrative autonomy:

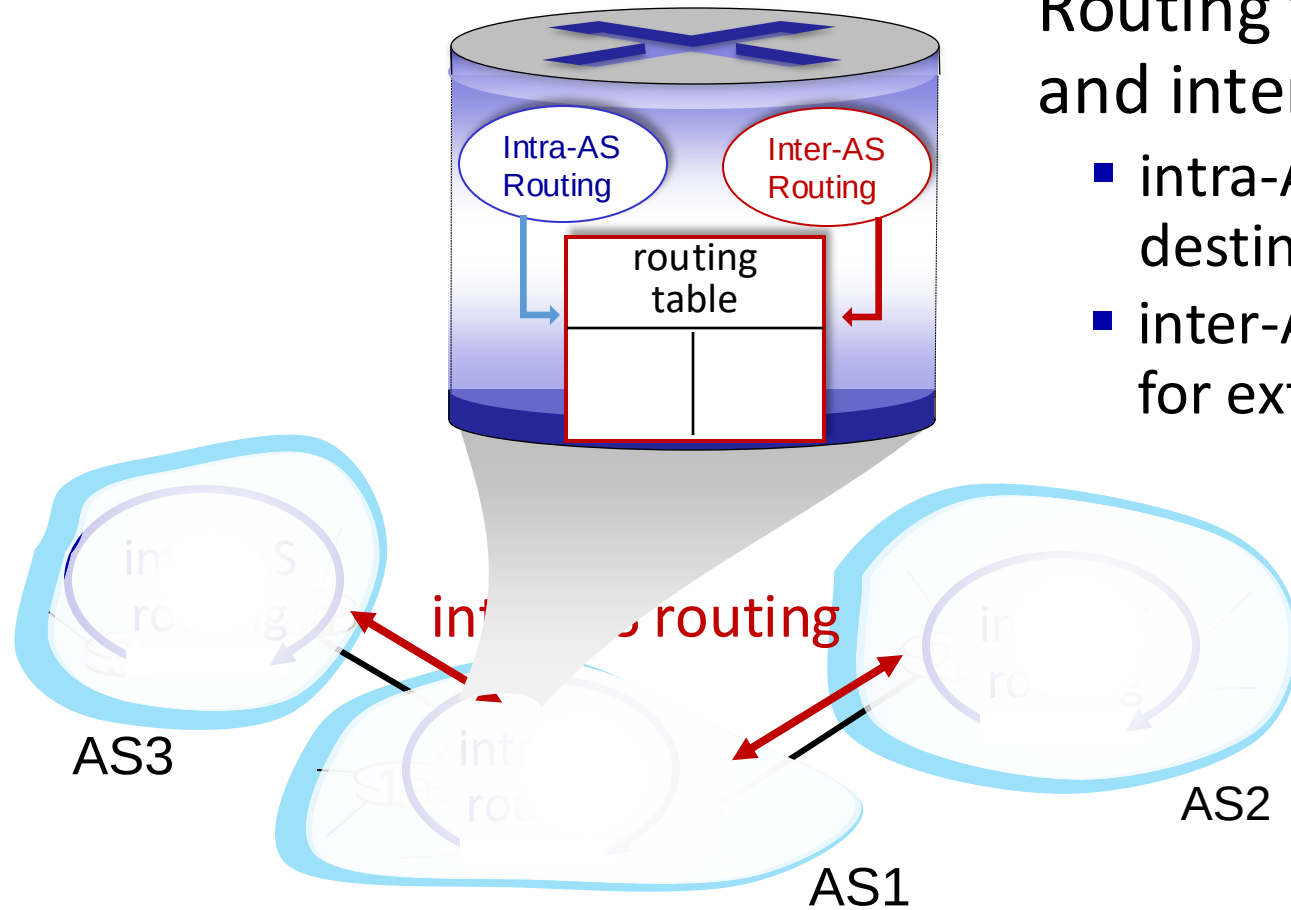
- Internet: a network of networks
- each network admin may want to control routing in its own network
- regions known as “autonomous systems” (AS) (a.k.a. “domains”)

Routing on the Internet

- Two different variants of routing protocols – (a) **Intra-AS routing protocols**, and (b) **Inter-AS routing protocols**
- **Intra-AS (Intra Domain) Routing Protocols:** Routing within an AS; Example- Routing Information Protocol (RIP), Open Shortest Path First (OSPF). Sometimes, they are called as **Interior Gateway Protocols (IGP)**
- **Inter-AS (Inter Domain) Routing Protocols:** Routing among the various ASes based on peering relationship; Example: Border Gateway Protocol (BGP). These are known as **Exterior Gateway Protocols (EGP)**

Gateway router: at “edge” of its own AS, has link(s) to router(s) in other AS'es

Interconnected AS



Routing table configured by intra- and inter-AS routing algorithms

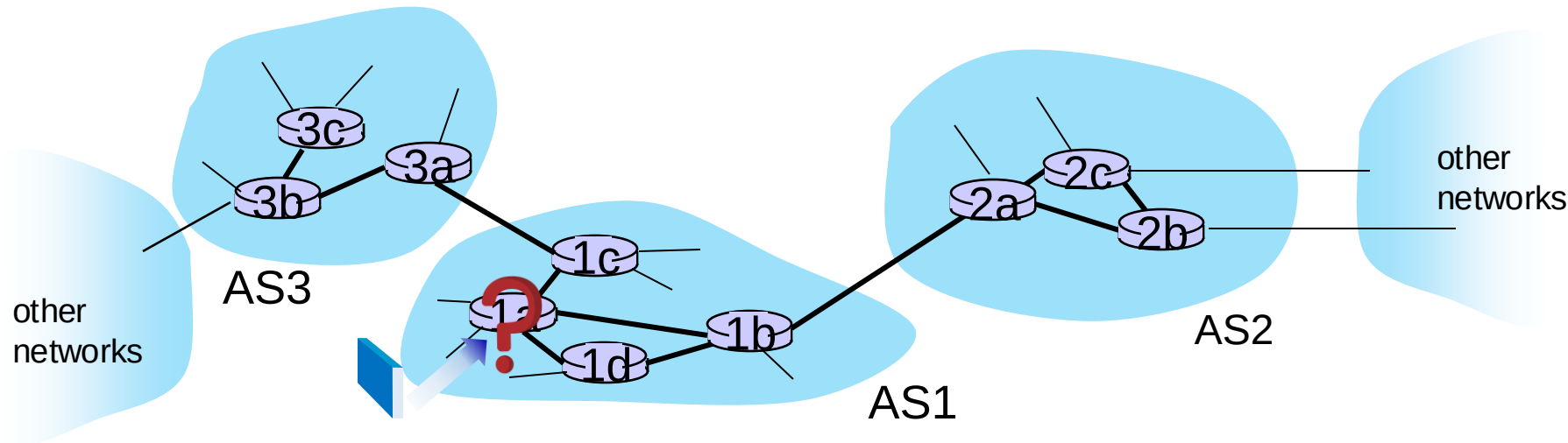
- intra-AS routing determine entries for destinations within AS
- inter-AS & intra-AS determine entries for external destinations

Inter-AS routing: a role in intradomain forwarding

- suppose router in AS1 receives datagram destined outside of AS1:
- ? • router should forward packet to gateway router in AS1, but which one?

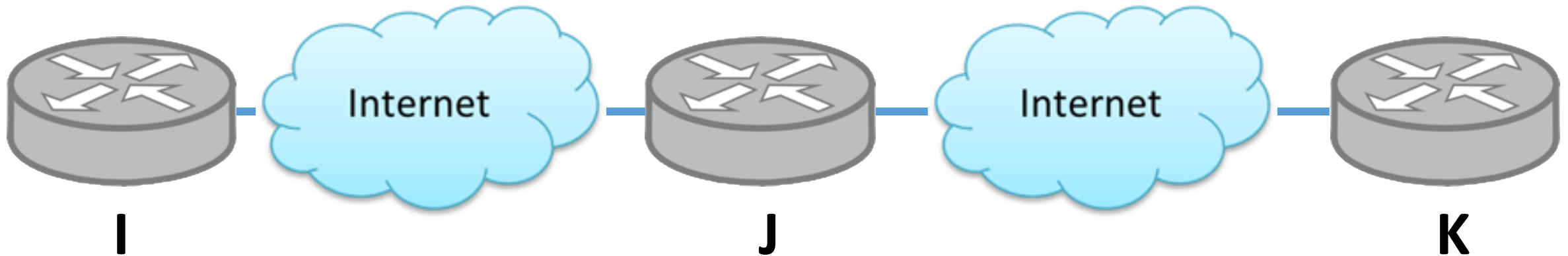
AS1 inter-domain routing must:

1. learn which destinations reachable through AS2, which through AS3
2. propagate this reachability info to all routers in AS1



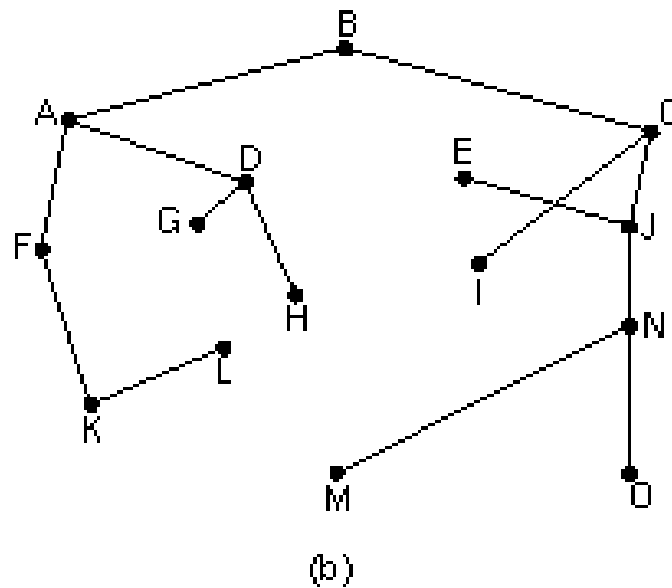
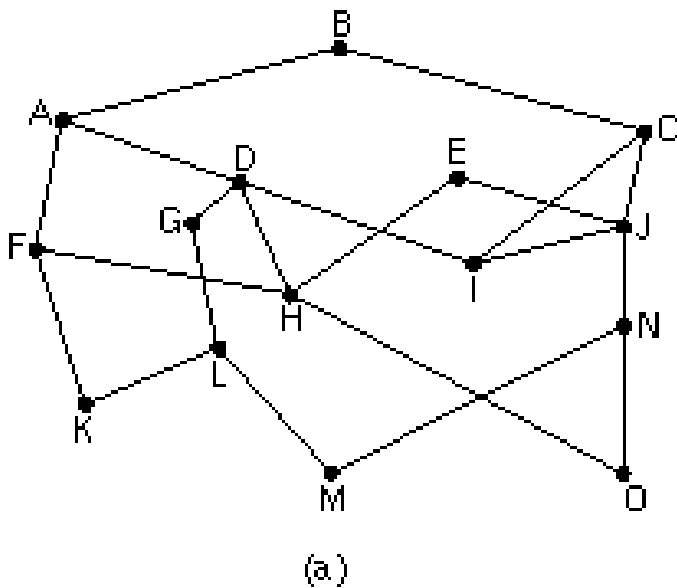
The Optimality Principle (Bellman 1957)

If Router J is on the optimal path from Router I to Router K, the the optimal path from J to K also falls among the same route.



Sink Tree

The set of optimal routes from **all sources** to a **given destination** form a tree **rooted at the destination**. Such a tree is called a **sink tree**.



Can you tell, why we are more focused to a destination centric tree, rather than a source centric tree?

Sink Tree

- Sink tree for a destination may not be unique – there can be an alternate path between two routers with the same path length.
- We may use a more generic data structure – a **Directed Acyclic Graph (DAG)**, denoting all possible paths from all the source to a destination.
- Both a tree or a DAG is okay – our objective here is to ensure **loop-free routing**

Distance Vector Routing

- Operates by having each router maintain a table (i.e., a vector) giving the best known distance to each destination and which link to use to get there.
- A distributed version of the **Bellman-Ford Algorithm for Shortest Path**.
- Used in **Routing Information Protocol (RIP)** – the original ARPANET routing algorithm.

Distance Vector Routing

- **Bellman Ford Equation (Dynamic Programming)**

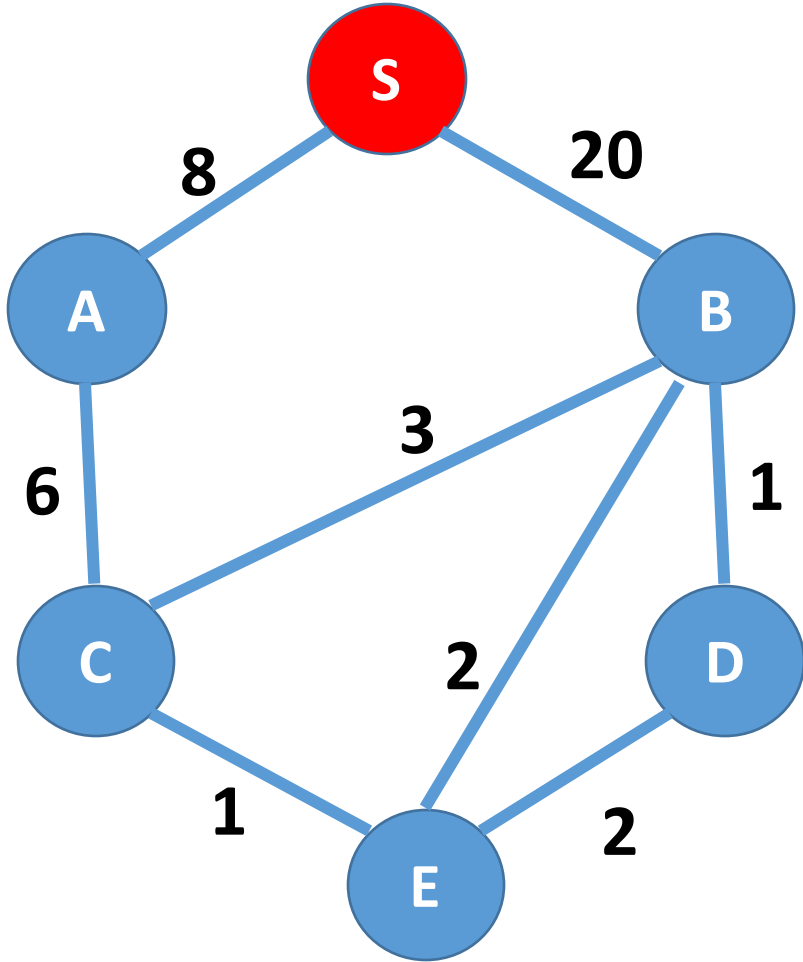
$$d_x(y) = \min\{c(x, v) + d_v(y)\}$$

$d_x(y)$: Cost of least cost path from node x to node y

$c(x, v)$: Cost from node x to node v where $v \in \mathbb{V}$, the set of neighbors for node x

- We start from the sink (the destination), and recursively construct the shortest path from all the nodes in the subnet.

Bellman Ford in Centralized Network



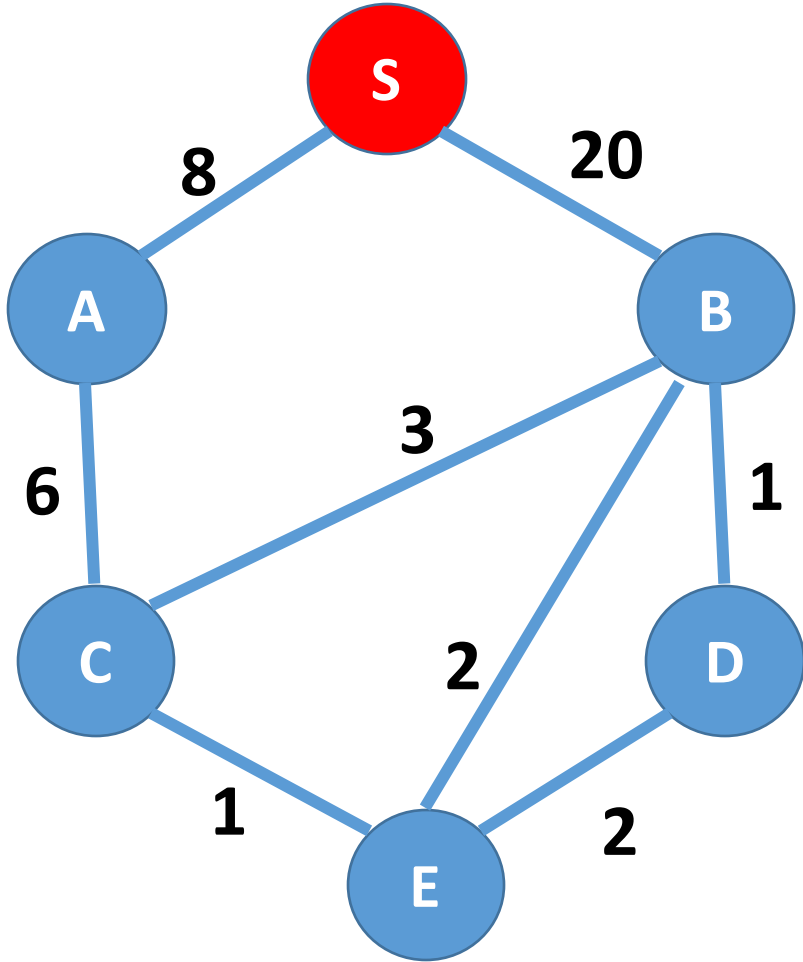
Initialize

S	A	B	C	D	E
0	8	20	∞	∞	∞
-	S	S	-	-	-

Iteration 1

S	A	B	C	D	E
0	8	20	14	21	22
-	S	S	A	B	B

Bellman Ford in Centralized Network



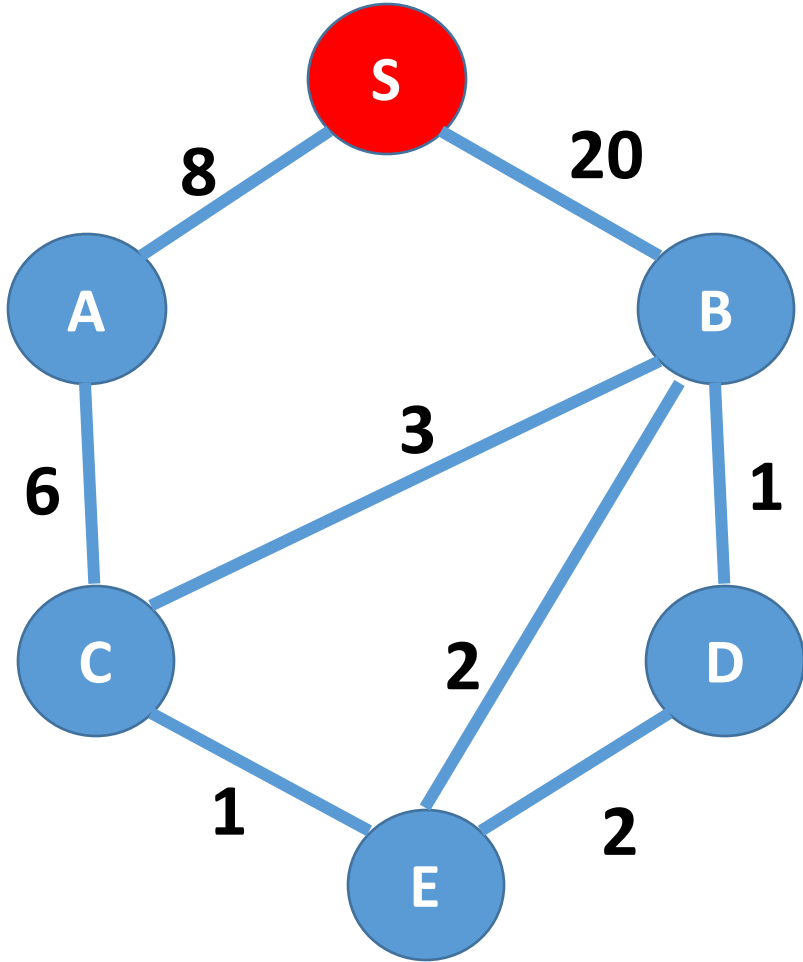
Iteration 1

S	A	B	C	D	E
0	8	20	14	21	22
-	S	S	A	B	B

Iteration 2

S	A	B	C	D	E
0	8	17	14	21	15
-	S	C	A	B	C

Bellman Ford in Centralized Network



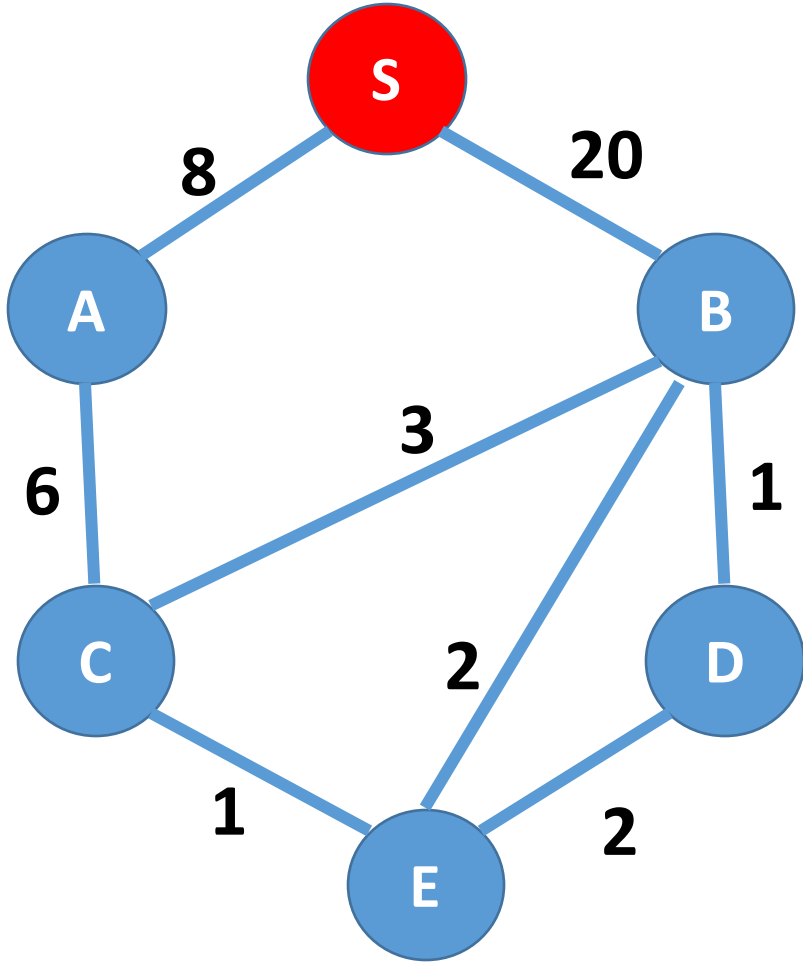
Iteration 2

S	A	B	C	D	E
0	8	17	14	21	15
-	S	C	A	B	C

Iteration 3

S	A	B	C	D	E
0	8	17	14	17	15
-	S	C	A	E	C

Bellman Ford in Centralized Network



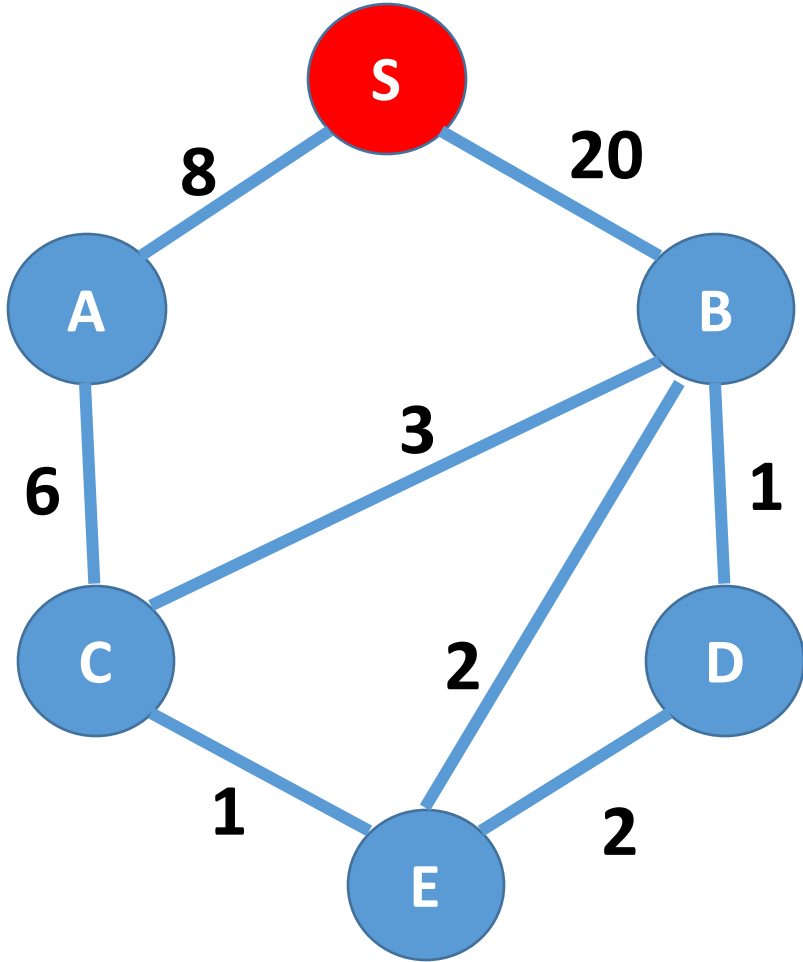
Iteration 3

S	A	B	C	D	E
0	8	17	14	17	15
-	S	C	A	E	C

Iteration 4

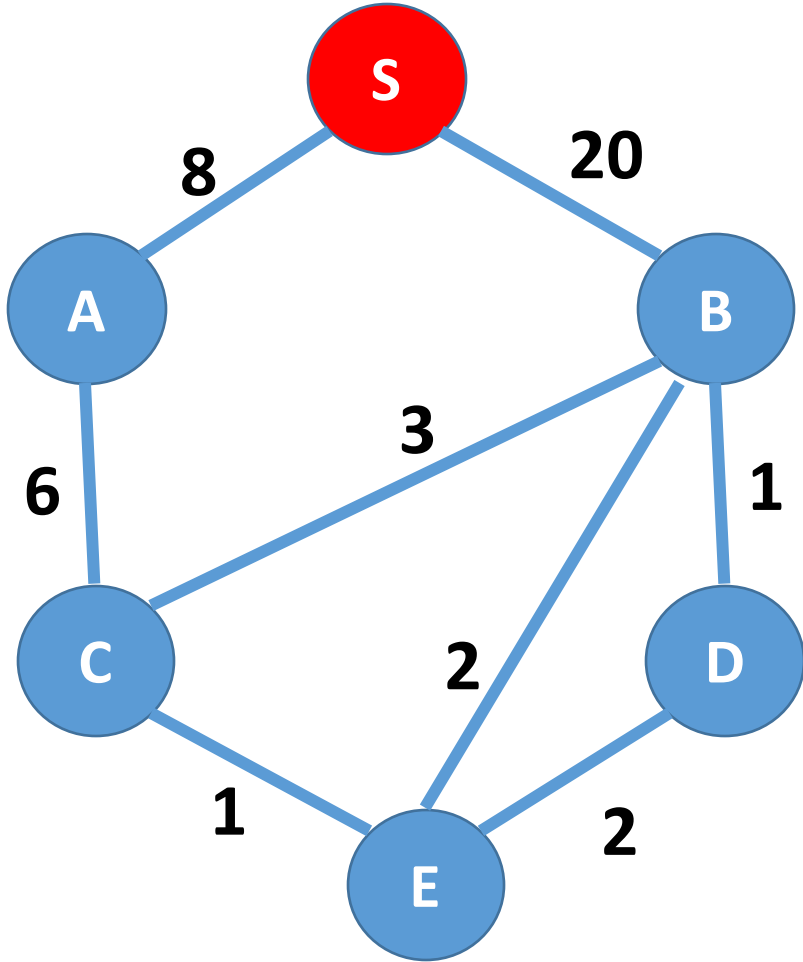
S	A	B	C	D	E
0	8	17	14	17	15
-	S	C	A	E	C

Distributed Bellman Ford To Distance Vector Routing



- How do we move from one iteration to the next iteration for Bellman Ford algorithm?
- We need the **currently computed distance information** from each node to others in the network – this is called the **distance vector**
- Used to find out the routes between any two nodes
- In RIP with CIDR, the routing table along with a metric field (denoting the cost) is used as the distance vector

Distributed Bellman Ford To Distance Vector Routing



	S	A		S
S	00,-	08,S		00,-
A	08,A	00,-		08,A
B	20,B	9,C		17,A
C	23,B	6,C		14,A
D	21,B	9,C		17,A
E	15,A	7,C		15,A

Centralized Bellman Ford Vs. Distance Vector Routing

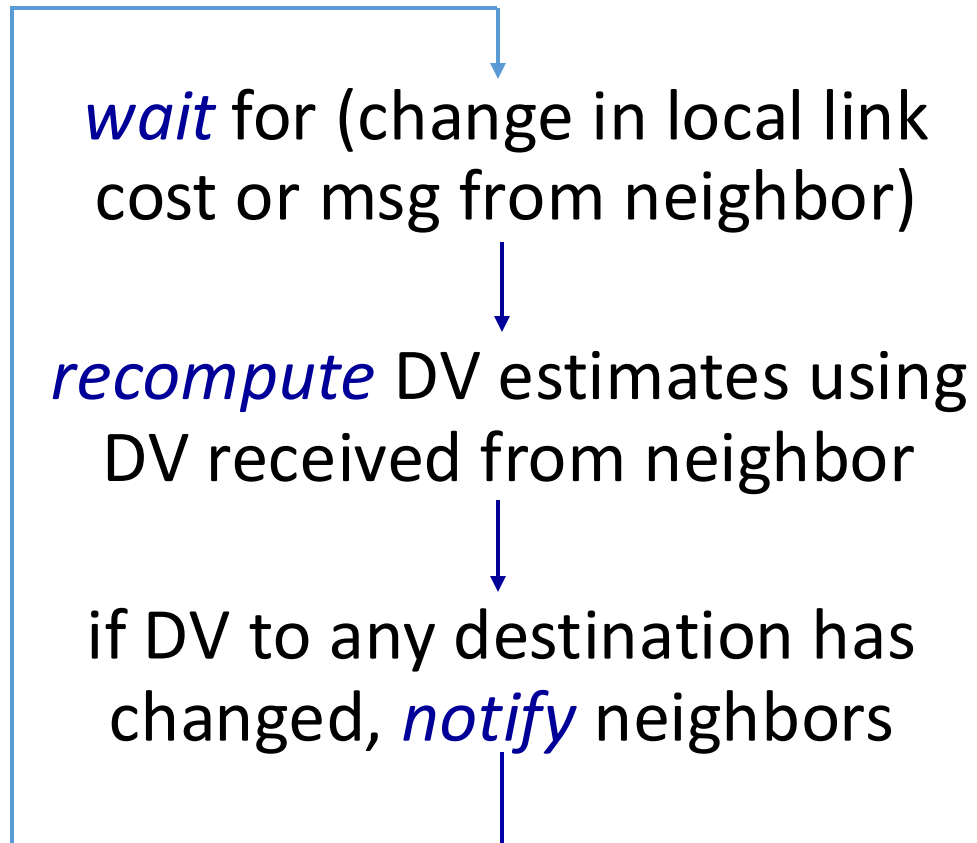
- Note the differences between **centralized Bellman Ford algorithm** and its distributed version used in **distance vector routing**.
- We are destination oriented (path to sink) for both the variants
- **Centralized Bellman Ford:** We find out the shortest path from all other nodes to the sink
- **Distance Vector Routing:** We find out the shortest path from the current node to all other nodes that can work as the sink
- **Can you guess the reason for this?**

Centralized Bellman Ford Versus Distance Vector Routing

- Can you guess the reason for this? – Think of the convergence criteria
- **Centralized Bellman Ford:** You know the number of iterations required. For n number of nodes, you need $n-1$ iterations.
 - **Requires synchronous updates of distance tables**
- **Distance Vector Routing:** You do not know the number of iterations required, number of nodes (n) changes dynamically, distance vector updates are asynchronous
 - **Update the local routing table asynchronously as and when you receive an update from a neighbor**

Distance vector algorithm

each node:



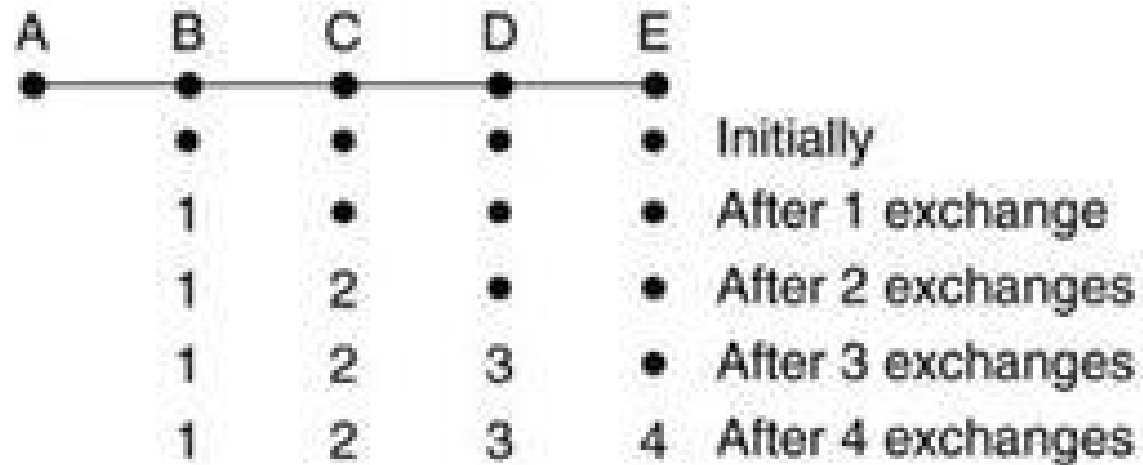
iterative, asynchronous: each local iteration caused by:

- local link cost change
- DV update message from neighbor

distributed, self-stopping: each node notifies neighbors *only* when its DV changes

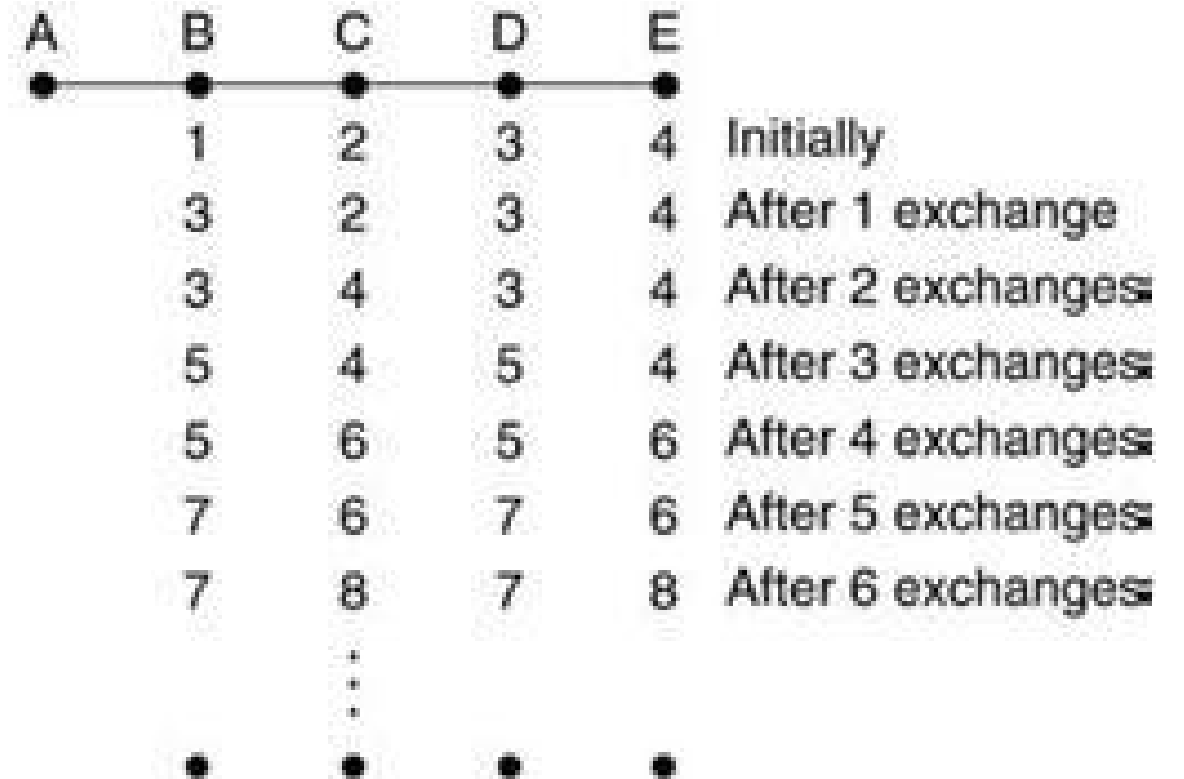
- neighbors then notify their neighbors – *only if necessary*
- no notification received, no actions taken!

The Count to Infinity Problem



(a)

Link A-B comes up



(b)

Link A-B gets down

Solving Count to Infinity

- **Split Horizon:** Routing information is prevented from exiting the router on an interface through which the information was received.
 - A router needs to wait for a timeout to remove an entry from a routing table
- **Poisson Reverse:** All known networks are advertised in each routing update. However, those networks learned through a specific interface are advertised as **unreachable** in the routing announcements sent out to that interface.
 - Routes are immediately removed when they are marked as unreachable from all the routing updates from all the neighbors.
- However, the solutions adds up cost – increases the size of the route updates (you need separate route broadcast at different interfaces)
 - **We do not use these in practice**

Limitations of Distance Vector Routing / RIP

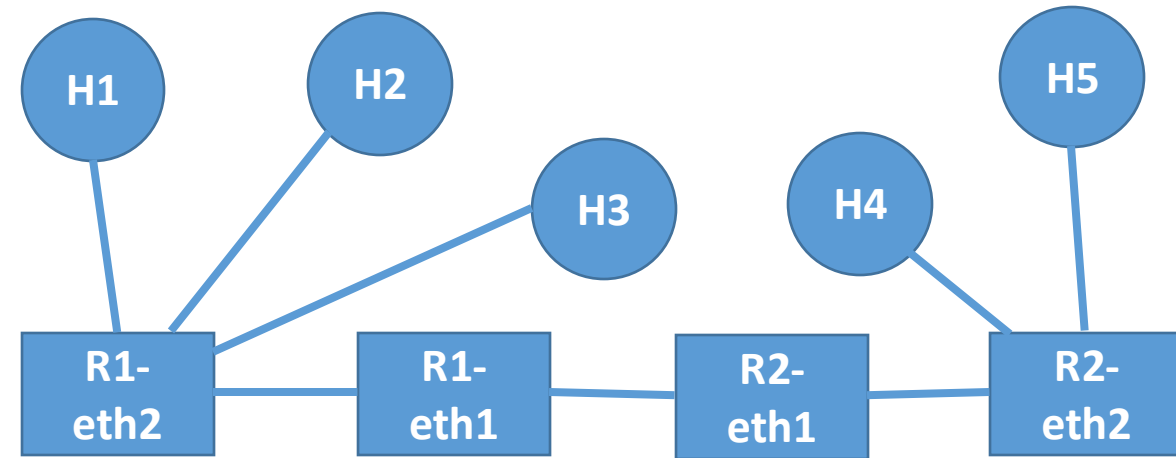
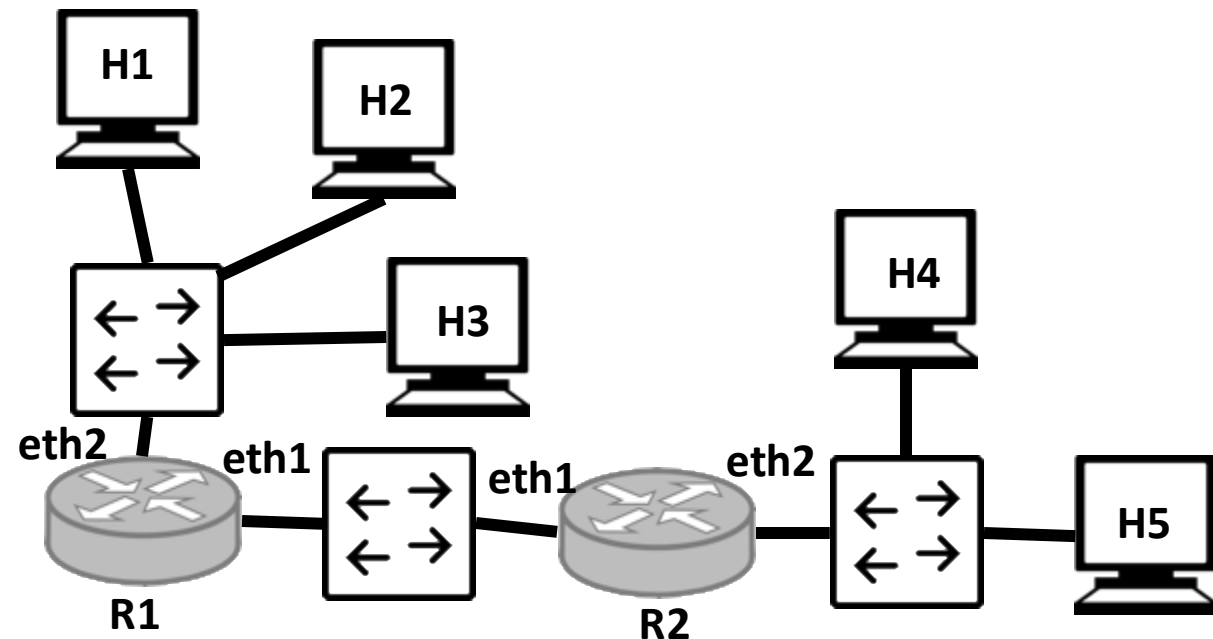
- The resolution to the counting to infinity problem enforces a maximum cost for a network path (generally 15 in RIP). This limits the diameter of a AS to a maximum of 15 hops.
- High signaling overhead - Periodic broadcasting of the distance vector table can result in increased utilization of the network resources for signaling.
- The algorithm is relatively slow to converge; you require information from all the nodes in the AS.

Link State Routing

- 1979: The ARPANET routing protocol was replaced by link state routing, as an impact to count-to-infinity problem (convergence become slow)
- The routing protocol – **Open Shortest Path First (OSPF)**
- The protocol is fairly simple
 - Discover neighbors and learn their network addresses
 - Set the distance or cost metric to each of the neighbors
 - Construct a packet telling all it has learned
 - Broadcast this packet – every router periodically learns the **link state** of the network graph
 - Compute the shortest path to every other routers

LSR - Learning about Routers

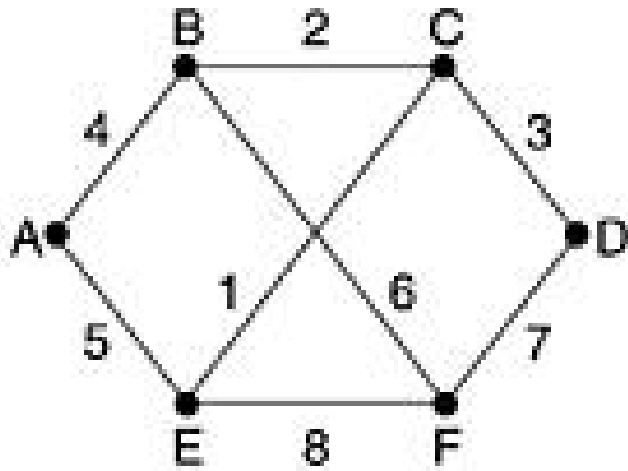
- When a router is booted, it first learn the neighbors – broadcast a HELLO packet on each point-to-point line – **note the use of broadcast IP address**
- Once a router receives a HELLO message, it sends back (this is a unicast IP packet) its identity (IP address)



LSR– Setting Link Costs

- Each link is assigned with a link cost or distance (hop count, delay) which is used as the routing metric to find out the shortest path
- A standard approach – inverse of the link bandwidth - higher capacity paths are better choices (minimize the routing cost)
- Some networks use link delay - computed through a ICMP ECHO packet from the IP layer
 - ICMP – **Internet Control Message Protocol** – a set of message suites for IP layer management functionalities
 - ICMP Echo Request and ICMP Echo Reply

LSR – Building Link State Packets



(a)

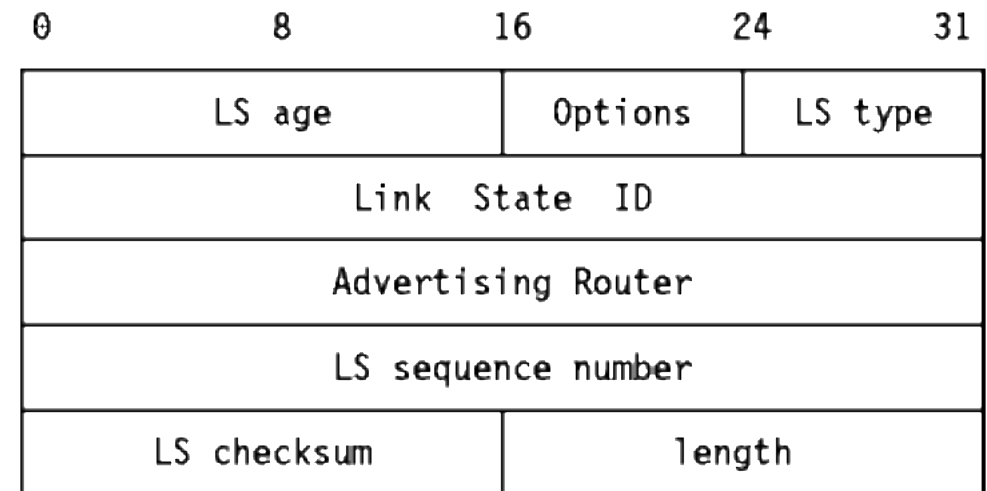
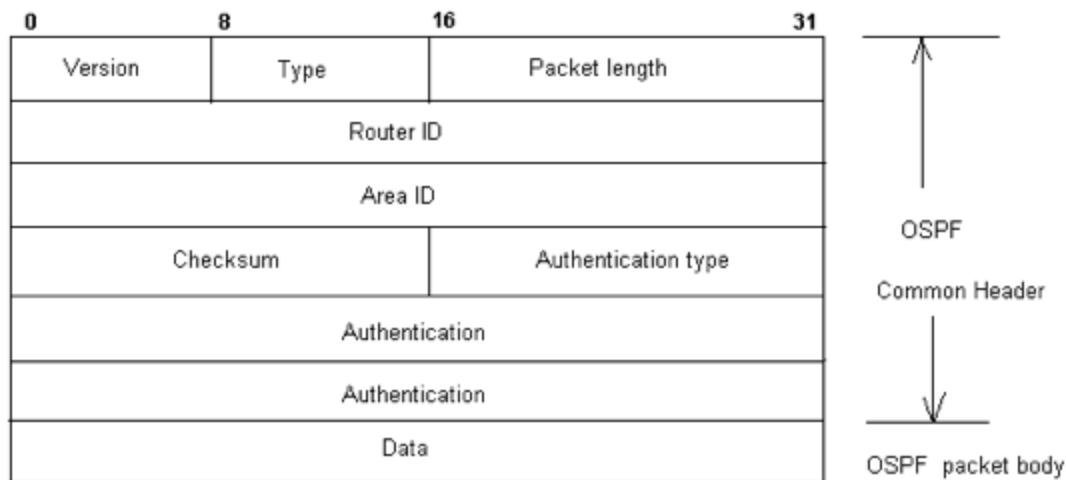
		Link	State		Packets	
A		B	C		D	
Seq.		Seq.	Seq.		Seq.	
Age		Age	Age		Age	
B	4	A	B	2	C	5
E	5	C	D	3	F	7
		F	E	1		
					A	5
					C	1
					F	8
					B	6
					D	7
					E	8

(b)

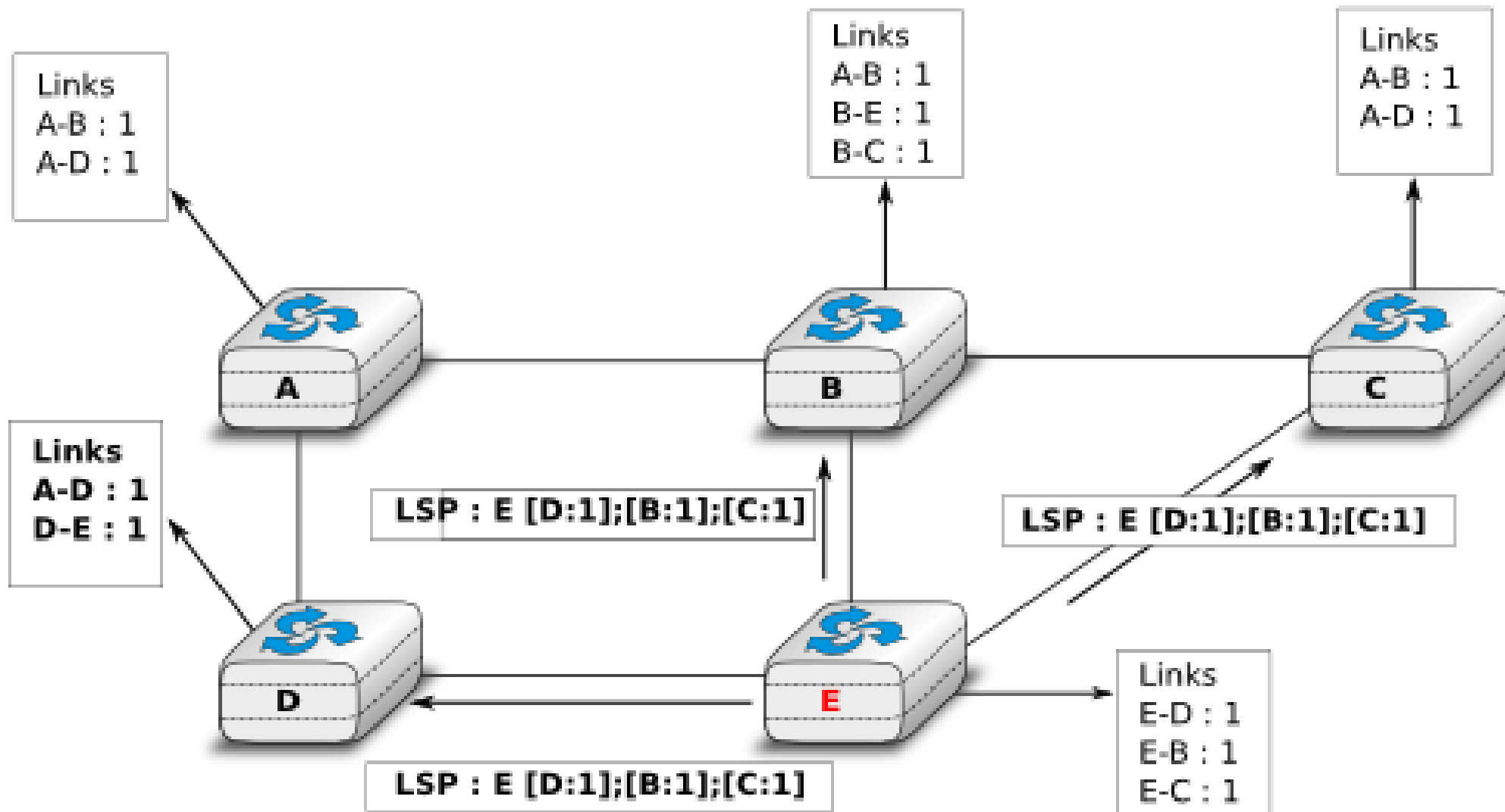
Image: Computer Networks, Andrew S. Tanenbaum, David J. Wetherall

LSR – Distributing the Link State Packets

- All the routers must get all the link state packets quickly and reliably
 - If different routers have different information, the routing inconsistency may occur
- Use flooding to distribute the link state packets to all the routers.
 - Each Link state packet contains a sequence number that is incremented for each new packet sent – **used to identify stale information**



LSR – Distributing the Link State Packets

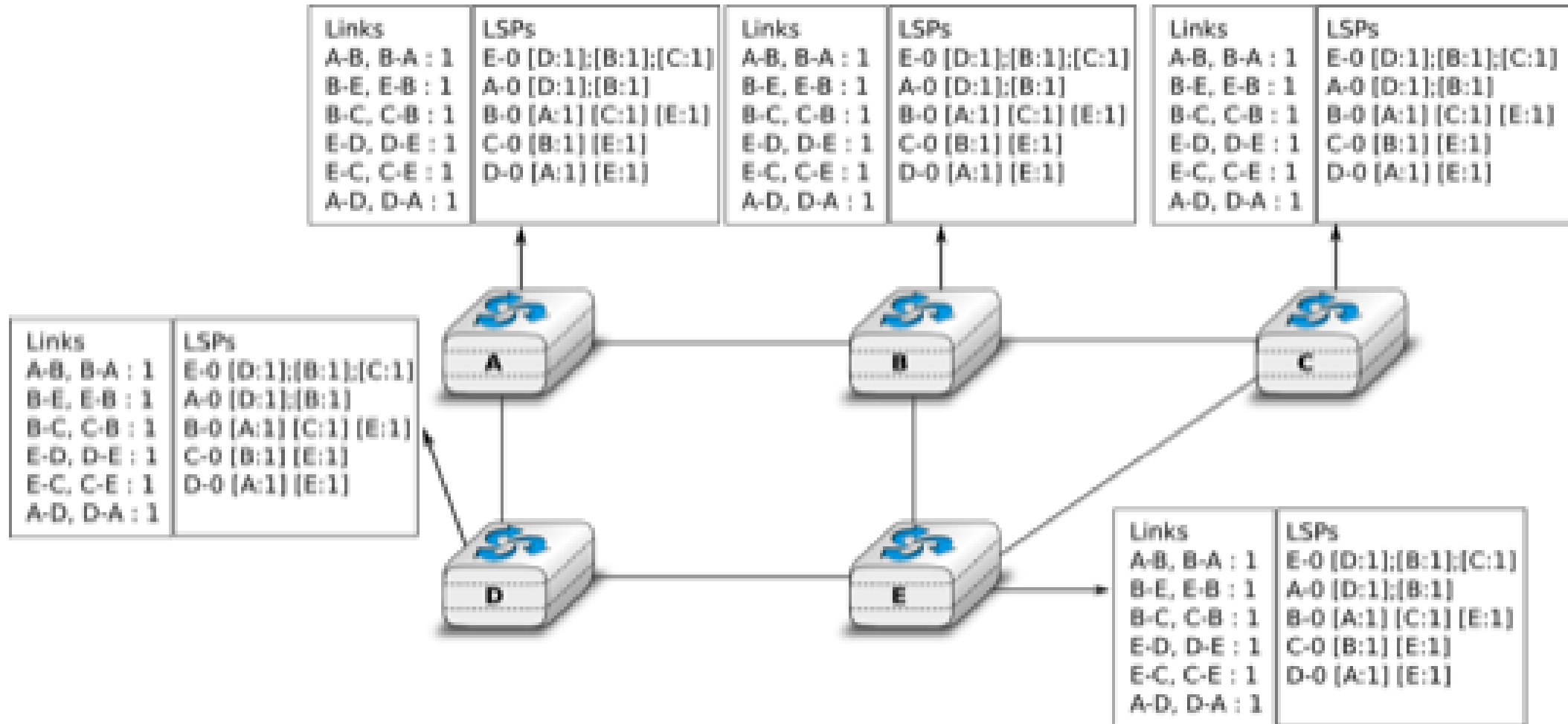


LSR – Distributing the Link State Packets

- Once a Link State (LS) packet is received, the sequence number is checked
 - If the sequence number is higher than the last observed LS packet, then it is accepted; otherwise, it is discarded
- **What if the sequence number wraps around?**
 - Use a 32-bit sequence number, and 1 LS packet per second – it would take 137 years to wrap around
- Every entry in the router is associated with an **age** – denotes the lifetime of an entry
 - Deletes the old entries from the routing table

LSR – Computing the New Routes

- Construct the network graph from the link state packets



Dijkstra's shortest path algorithm

- **centralized**: network topology, link costs known to *all* nodes
- all nodes have same info
- computes least cost paths from one node ("source") to all other nodes
 - gives *forwarding table* for that node
- **iterative**: after k iterations, know least cost path to k destinations

notation

- $c_{x,y}$: direct link cost from node x to y ; $= \infty$ if not direct neighbors
- $D(v)$: *current* estimate of cost of least-cost-path from source to destination v
- $p(v)$: predecessor node along path from source to v
- N' : set of nodes whose least-cost-path *definitively* known

Dijkstra's algorithm

1 *Initialization:*

2 $N' = \{u\}$ /* compute least cost path from u to all other nodes */

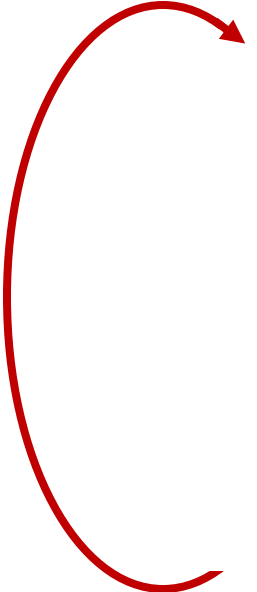
3 for all nodes v

4 if v adjacent to u /* u initially knows direct-path-cost only to direct neighbors */

5 then $D(v) = c_{u,v}$ /* but may not be *minimum* cost! */

6 else $D(v) = \infty$

7



8 *Loop*

9 find w not in N' such that $D(w)$ is a minimum

10 add w to N'

11 update $D(v)$ for all v adjacent to w and not in N' :

12 **$D(v) = \min (D(v), D(w) + c_{w,v})$**

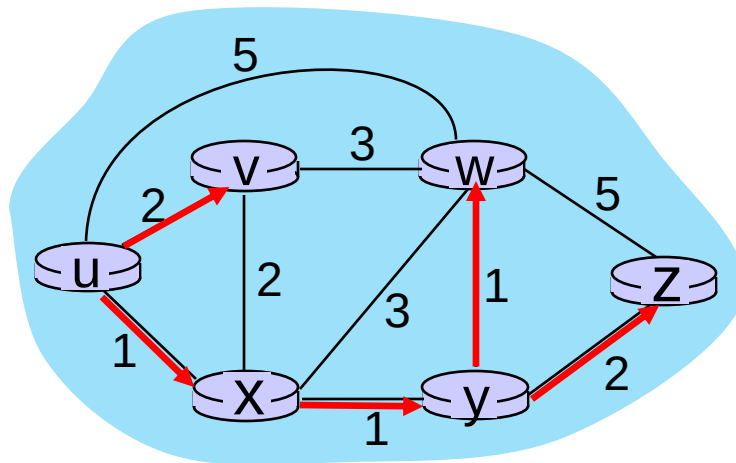
13 /* new least-path-cost to v is either old least-cost-path to v or known

14 least-cost-path to w plus direct-cost from w to v */

15 *until all nodes in N'*

Dijkstra's algorithm: an example

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	u, x	2, u	4, x		2, x	∞
2	u, x, y	2, u	3, y			4, y
3	u, x, y, v		3, y			4, y
4	u, x, y, v, w					4, y
5	u, x, y, v, w, z					



Initialization (step 0): For all a : if a adjacent to then $D(a) = c_{u,a}$

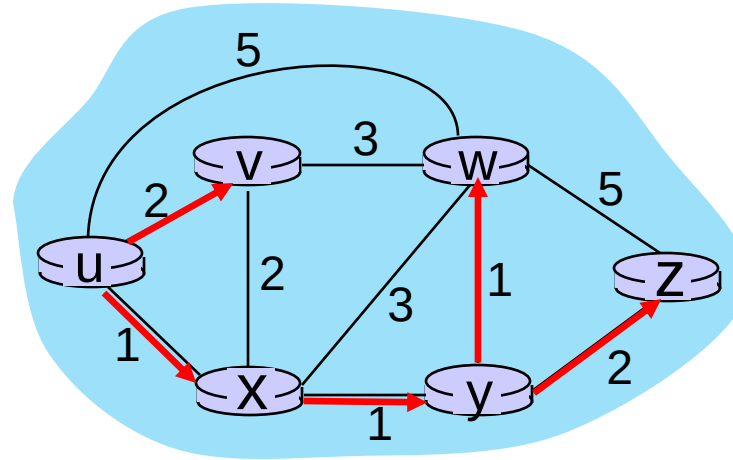
find a not in N' such that $D(a)$ is a minimum

add a to N'

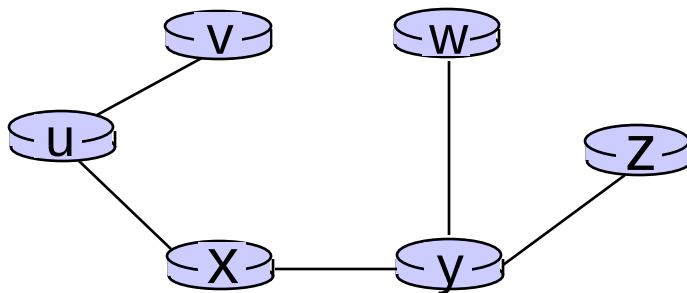
update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

Dijkstra's algorithm: an example



resulting least-cost-path tree from u:



resulting routing table in u:

destination	outgoing link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

route from u to v directly

route from u to all other destinations via x

Link State Routing - OSPF

- Link State Messages – **Overhead for the LSR**
- **OSPF Messages**
 - HELLO: Used to establish neighborhood
 - Database Descriptor (DD or DBD): Broadcast the local routing database among neighbors – check consistency of database information among routers
 - Link State Requests (LSR): Explicitly requests for link state information based on the database comparison
 - Link State Updates (LSU): Forward link state information
 - Link State Acknowledgements (LSAck): Acknowledges the receipt of link state information

Inter-domain Routing Protocols

- Routing between two autonomous systems (AS) – we call them as **routing domains**
 - **Exterior Gateway Protocol (EGP)**
- **Multi-homed AS:** has connection to more than one AS
- **Stub AS:** Connected to only one other AS
- **Transit AS:** Provides connection to other AS

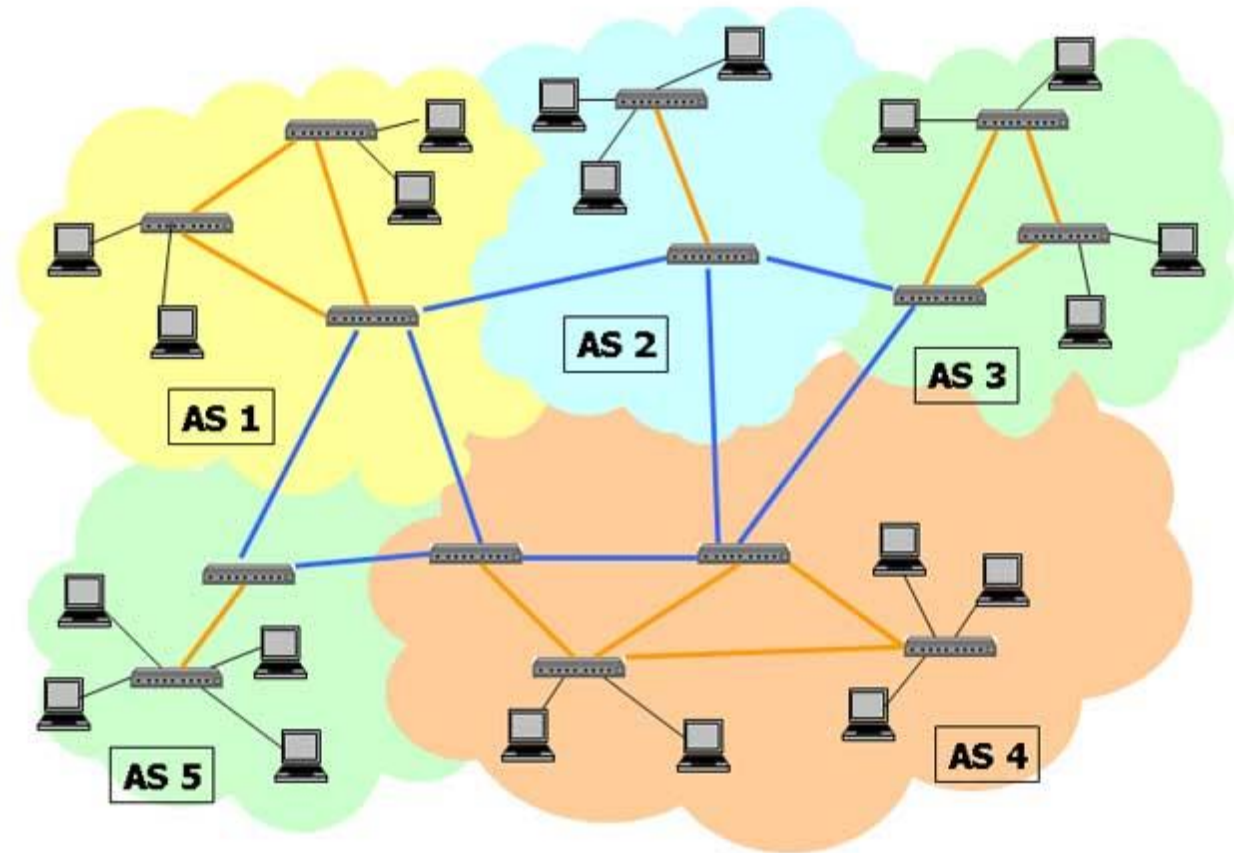
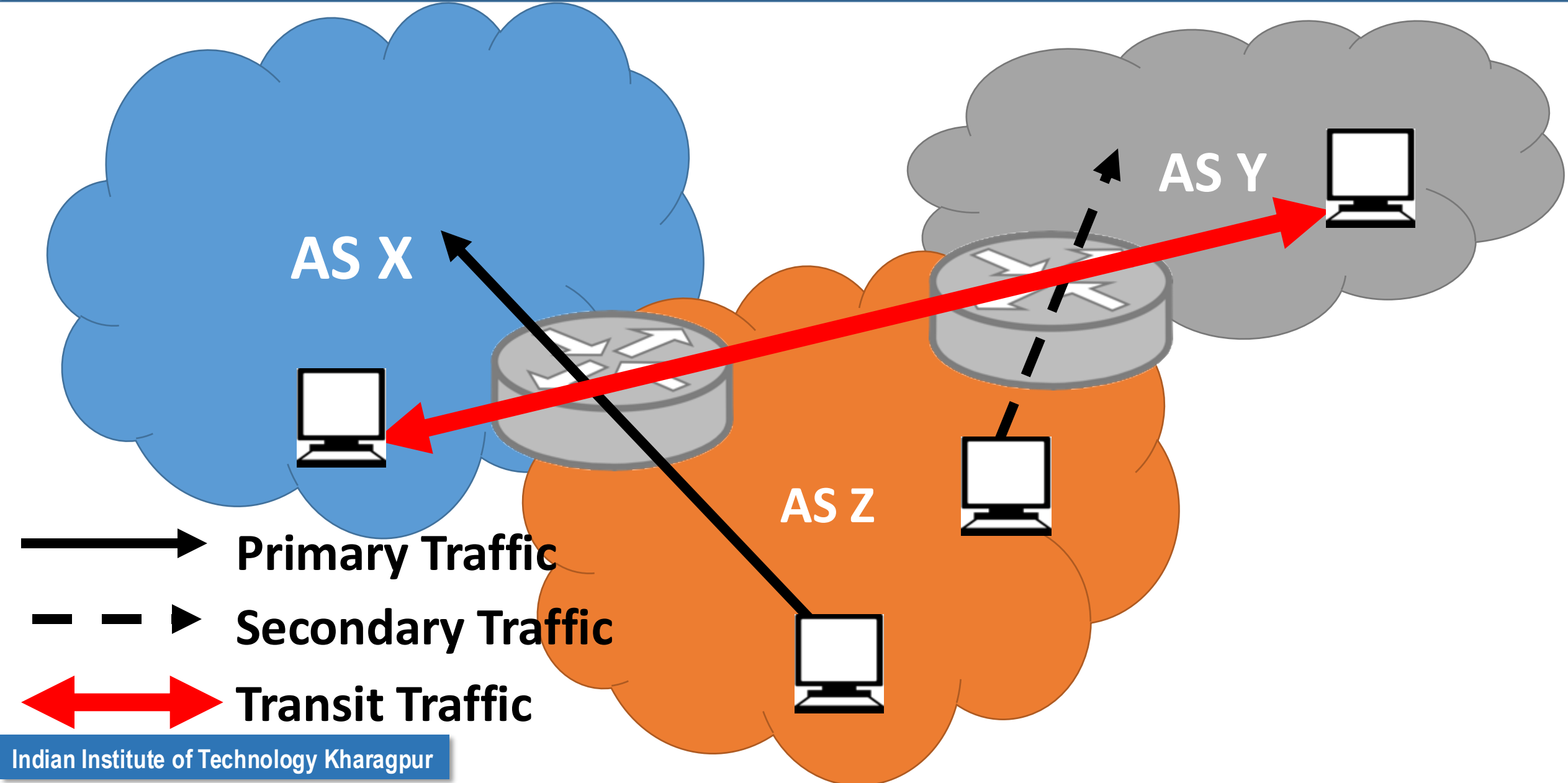


Photo courtesy: <http://www.web3.lu/>

Routing Protocols inside and across AS

- Each AS can run its own intra-domain routing protocols, we call them as **interior gateway protocols (IGP)**
 - Open Shortest Path First (OSPF)
 - Routing Information Protocol (RIP)
 - Can even use static routing or a mixed of IGPs at different subnets
- Inter-domain routing problem – **the AS shares *reachability information*** – description of the set of IP addresses that can be reached via a given AS
- **Challenge** – Each AS has to determine its own ***routing policies*** (can be complex)
 - *Whenever possible, I prefer to send traffic via AS X than via AS Y, but I'll use AS Y if it is the only path; and I never want to carry traffic from AS X to AS Y or vice versa.*

Routing Policies in the Internet

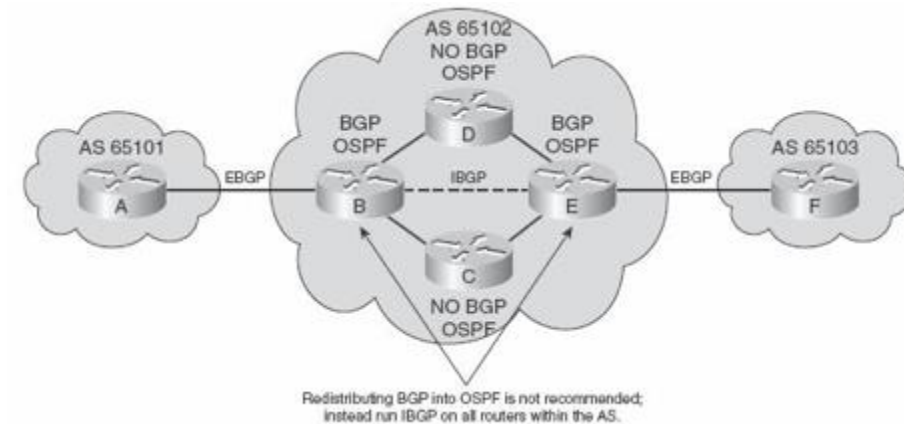
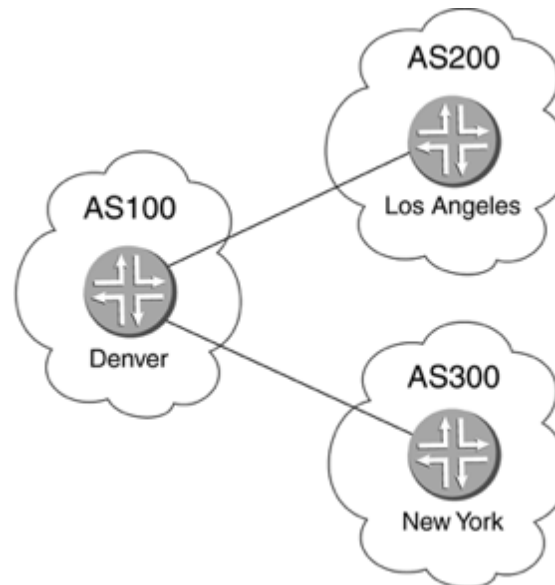
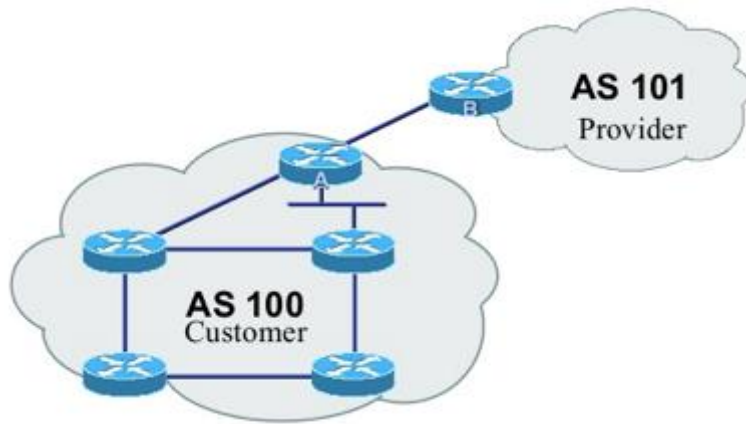


Border Gateway Protocol (BGP)

- The initial protocol (called EGP) was designed for specialized topology, such as a tree topology.
- BGP replaces EGP – generalizes the topology structure of the Internet.
- BGP assumes that the Internet is an arbitrary interconnected set of ASs.
- **Local Traffic:** Originates at or terminates on nodes within an AS
- **Transit Traffic:** Traffic that passes through an AS

Traffic through an AS

- **Stub AS:** Only carry local traffic
- **Multi-homed AS:** Only carry local traffic, refuses to carry transit traffic
- **Transit AS:** Carry both transit and local traffic



Objectives of BGP

- **Best non-looping policy-complaint path**
 - Loop free path through the ASs
 - Complaint with the policies of the various ASs along the path
- **Scaling** – CIDR at the Internet scale may not be scalable – You need to store IP/netmask information for thousands of subnets
 - Define paths by AS numbers, not the IP; Example AS12-AS14-AS76-AS132-AS45-AS61
- **Path Cost** – the autonomous systems are “*autonomous*” – every AS has their own interior routing protocol and own routing metrics – how to define a path metric for a BGP path?
 - Advertise only the ***reachability*** information, no path costs are associated with

Network Layer Reachability Information (NLRI)

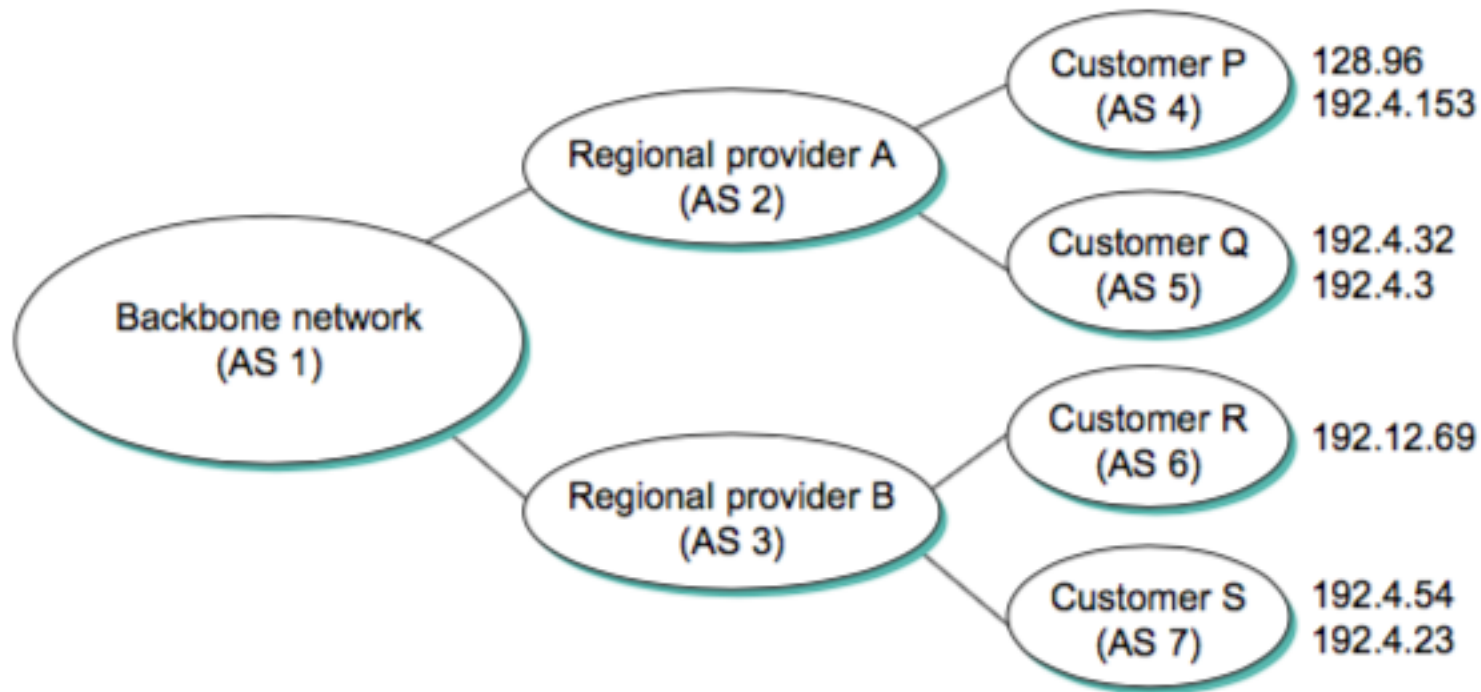
- AS Reachability information shared by the BGP supported routers (called the BGP peers)
- Shared between BGP peers through BGP UPDATE messages
- An IPv4 prefix (with IPv4 protocol) with the corresponding network IP
 - Example: 110.12, /16
- Used to find out the AS reachability as well as route aggregation through CIDR
 - Combine 110.12/16 and 110.12.8/24 together to form 110.12/16 – You do not need to maintain duplicate paths

BGP Speakers and BGP Peers

- **BGP Speakers** – A router configured with BGP – a spoke-person for the entire AS
 - Advertises the reachability information for this AS
- Once initialized, uses the well known BGP port (TCP port 179) to connect to other configured BGP peers in the Internet, and share the AS reachability information
- BGP speakers advertised the path information with the BGP speakers in the peer ASs.

BGP Path Information

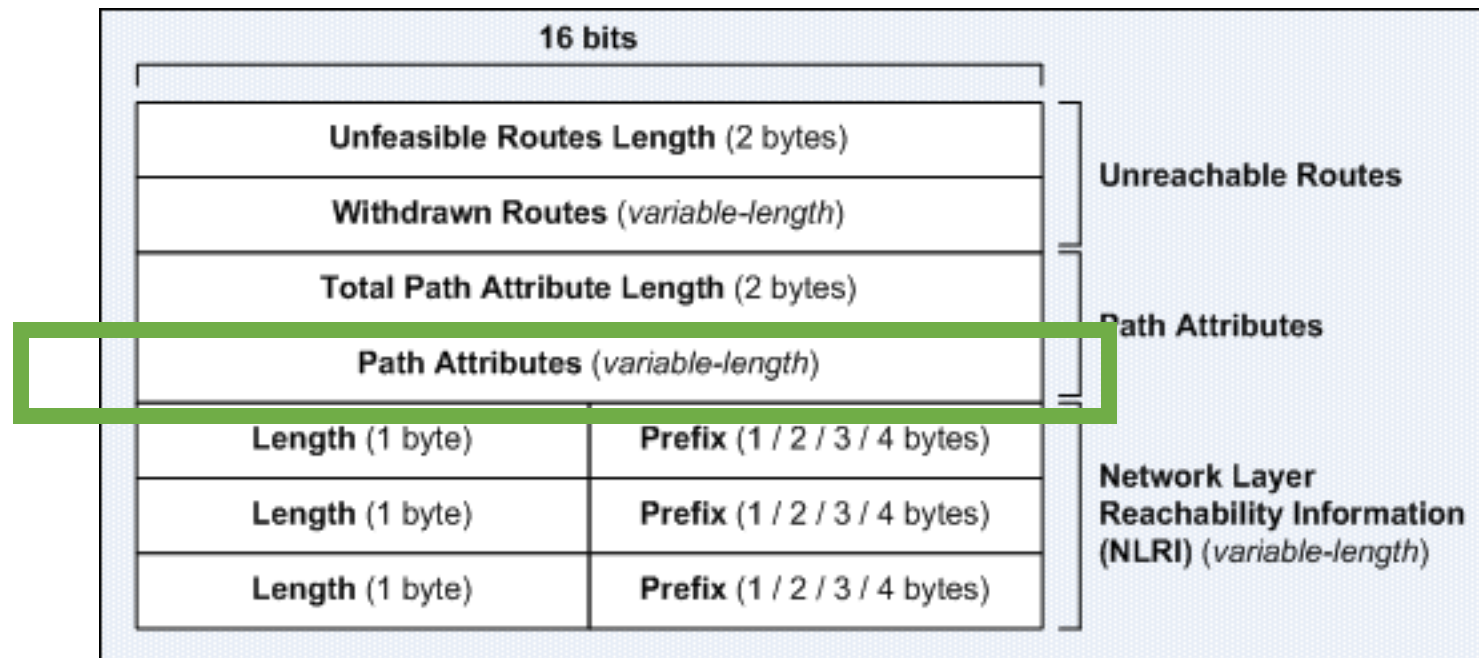
- BGP advertises ***complete path*** information as enumerated list of ASs to reach a particular network
 - Necessary to enable the sorts of policy decisions
 - Enables routing loops to be readily detected



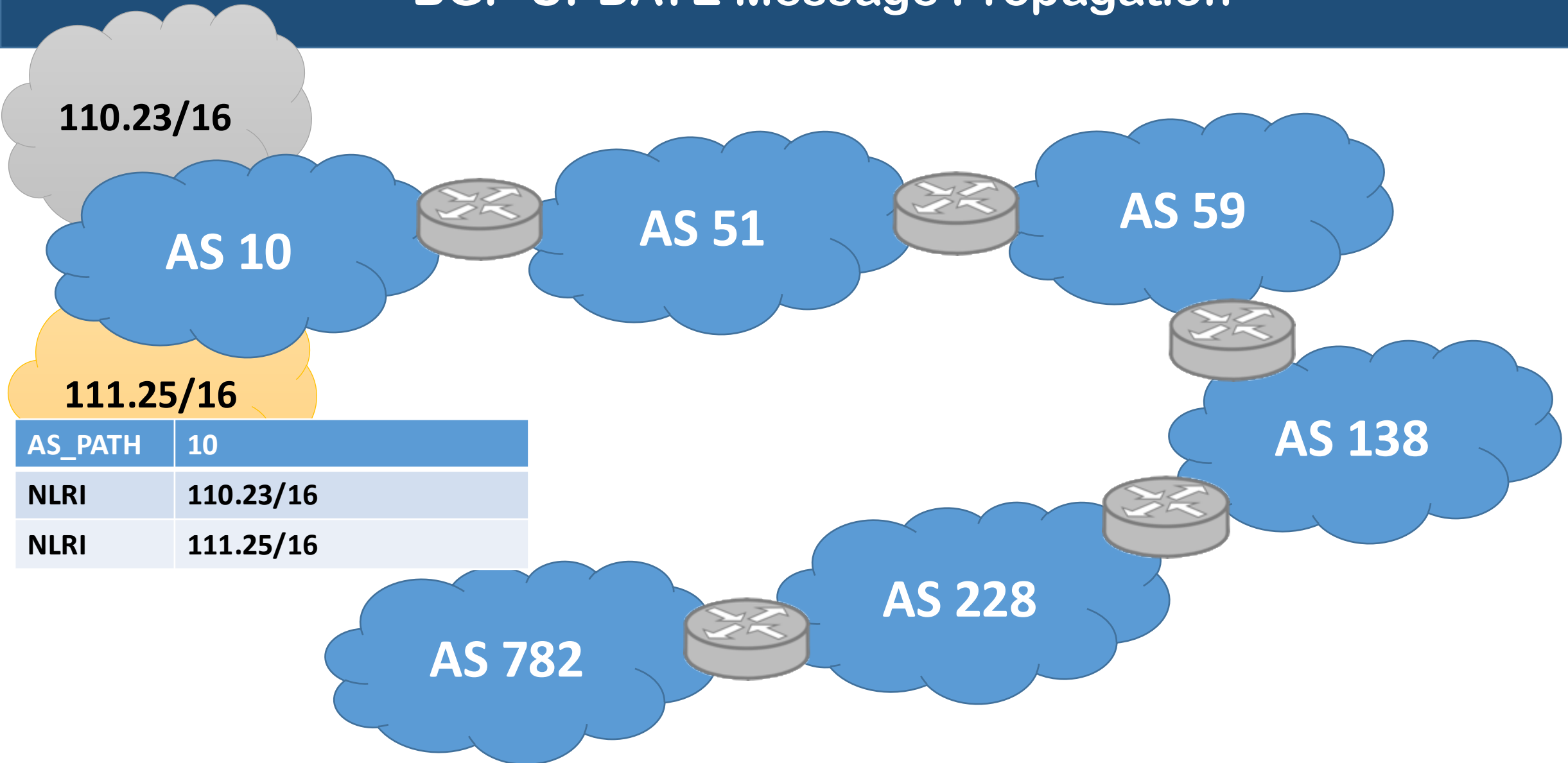
- AS 2 can advertise NLRI for the subnets given to Customer P and Customer Q
- AS 3 can reach to the subnet 128.96 via the AS Path AS3-AS1-AS2-AS4

BGP AS_PATH Attribute in UPDATE Message

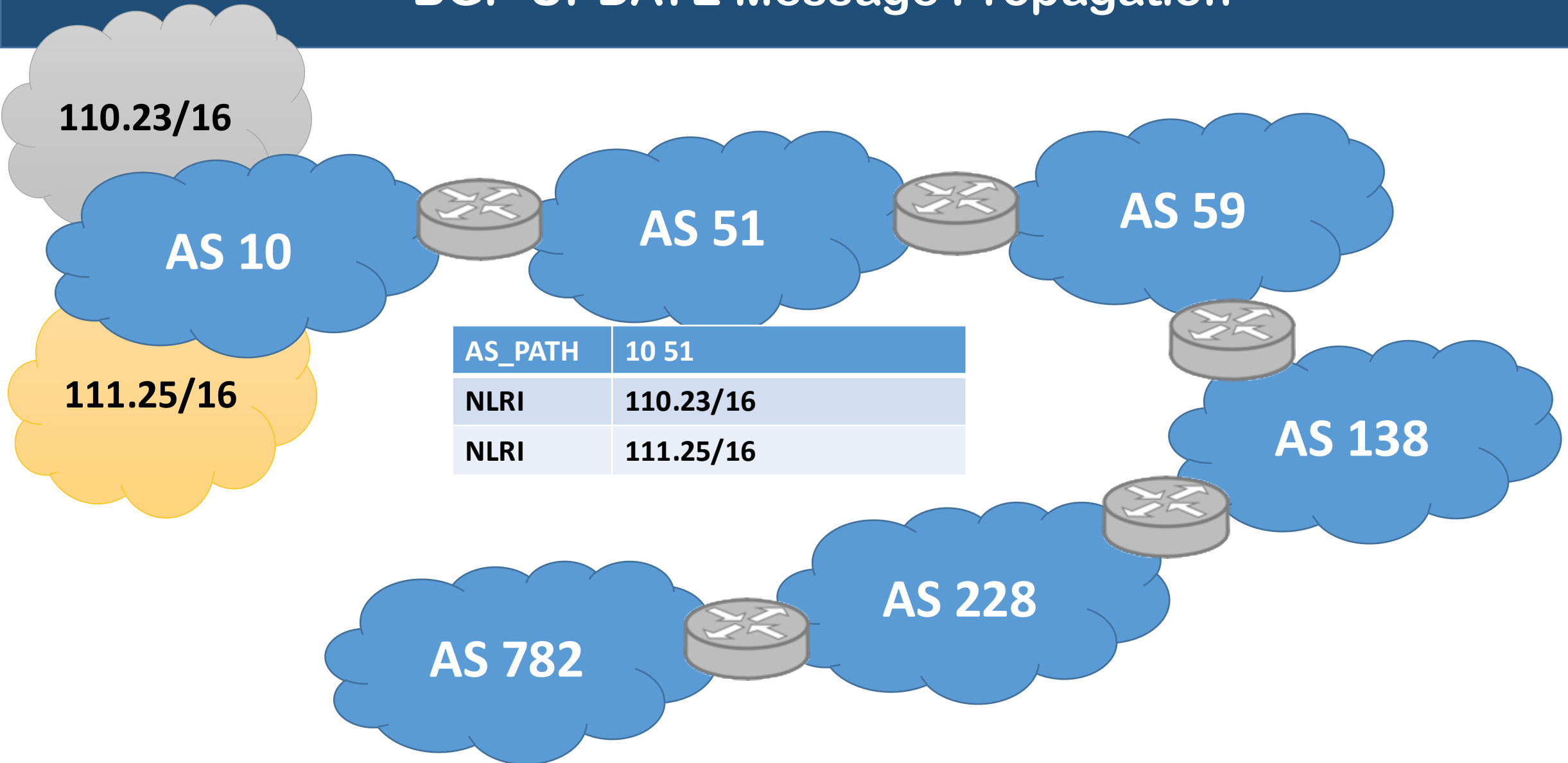
- Stores all the paths across various ASs through which a BGP UPDATE message has passed
- Every BGP speaker, when receives an UPDATE message, appends its own AS number and advertise that to the BGP peers



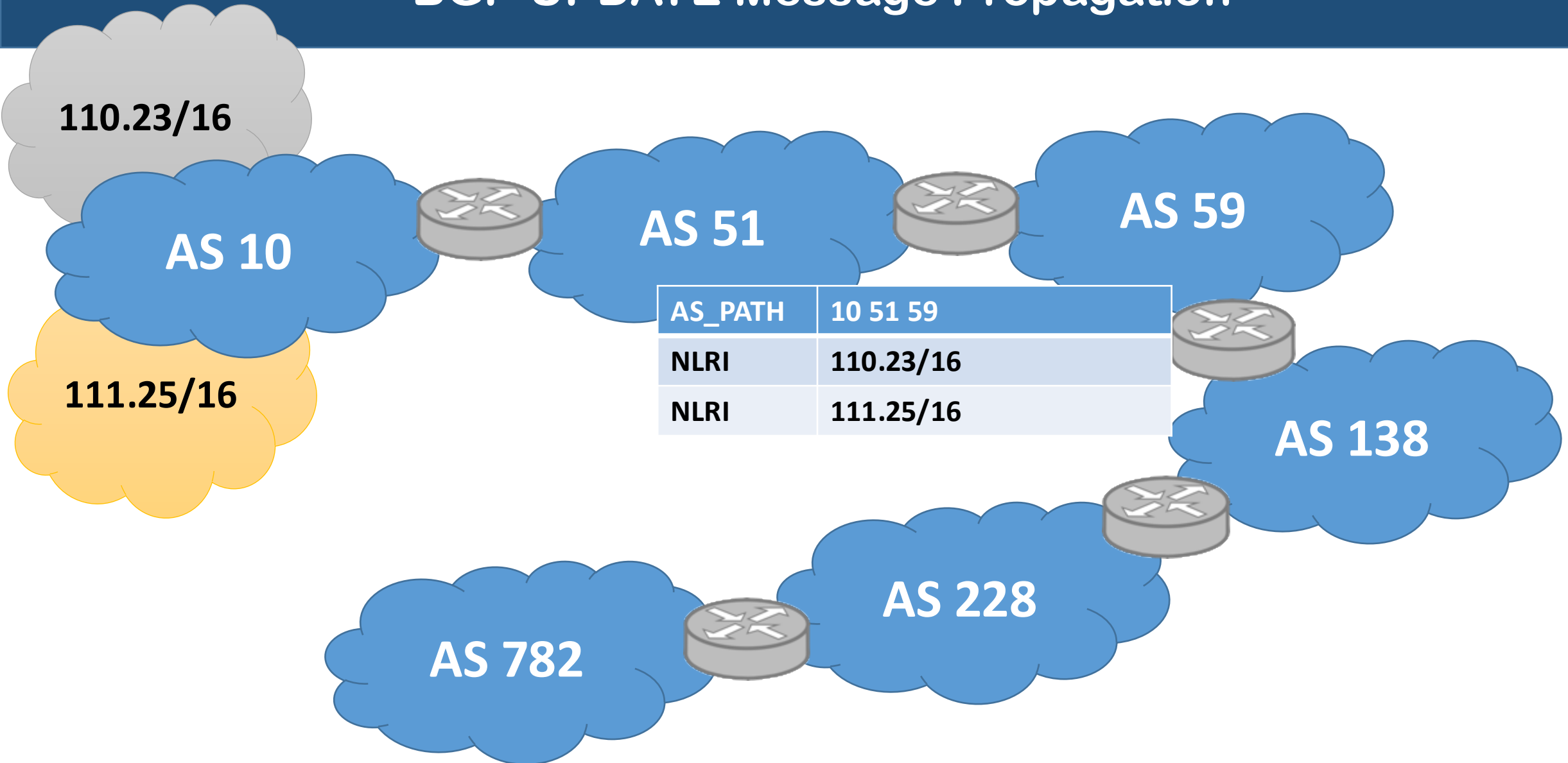
BGP UPDATE Message Propagation



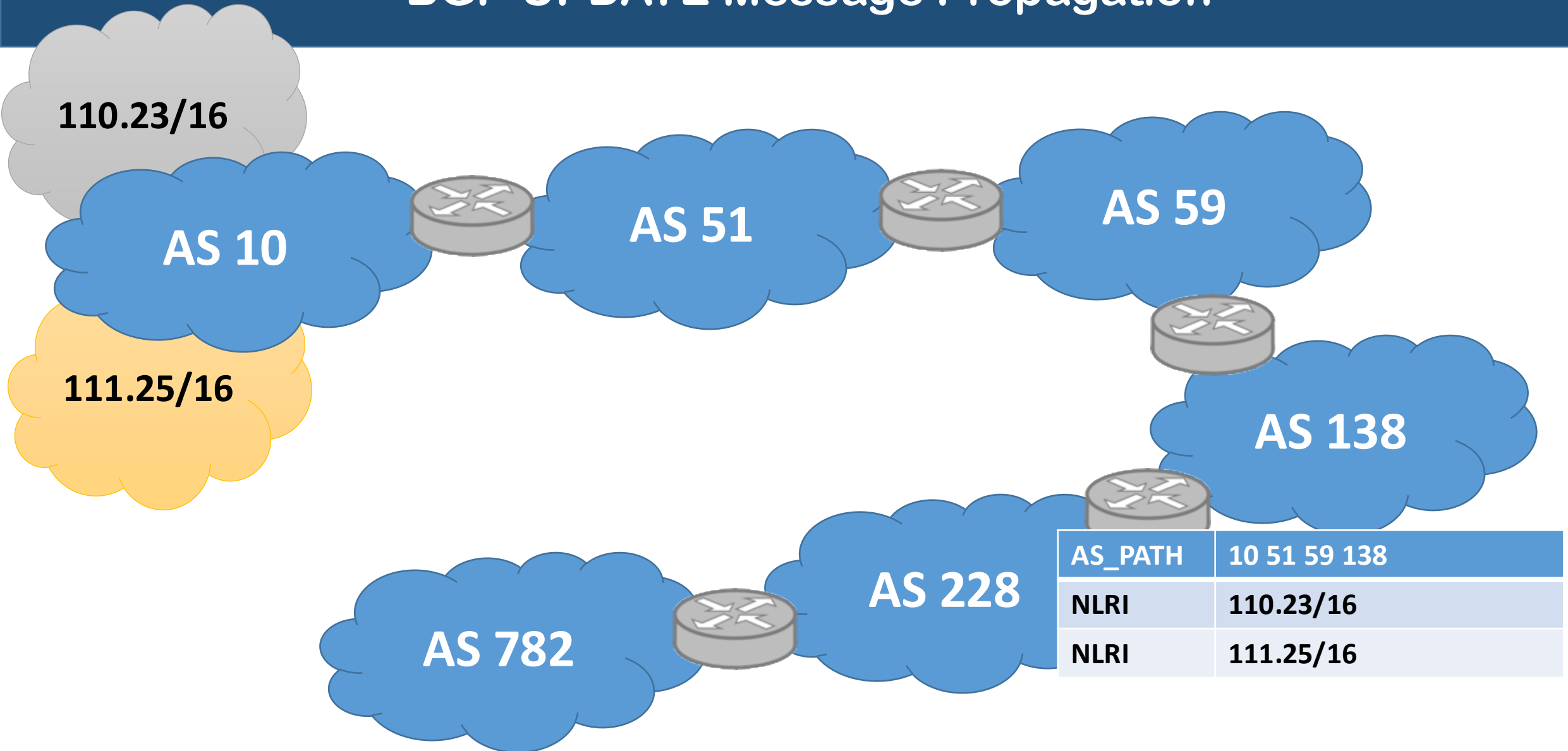
BGP UPDATE Message Propagation



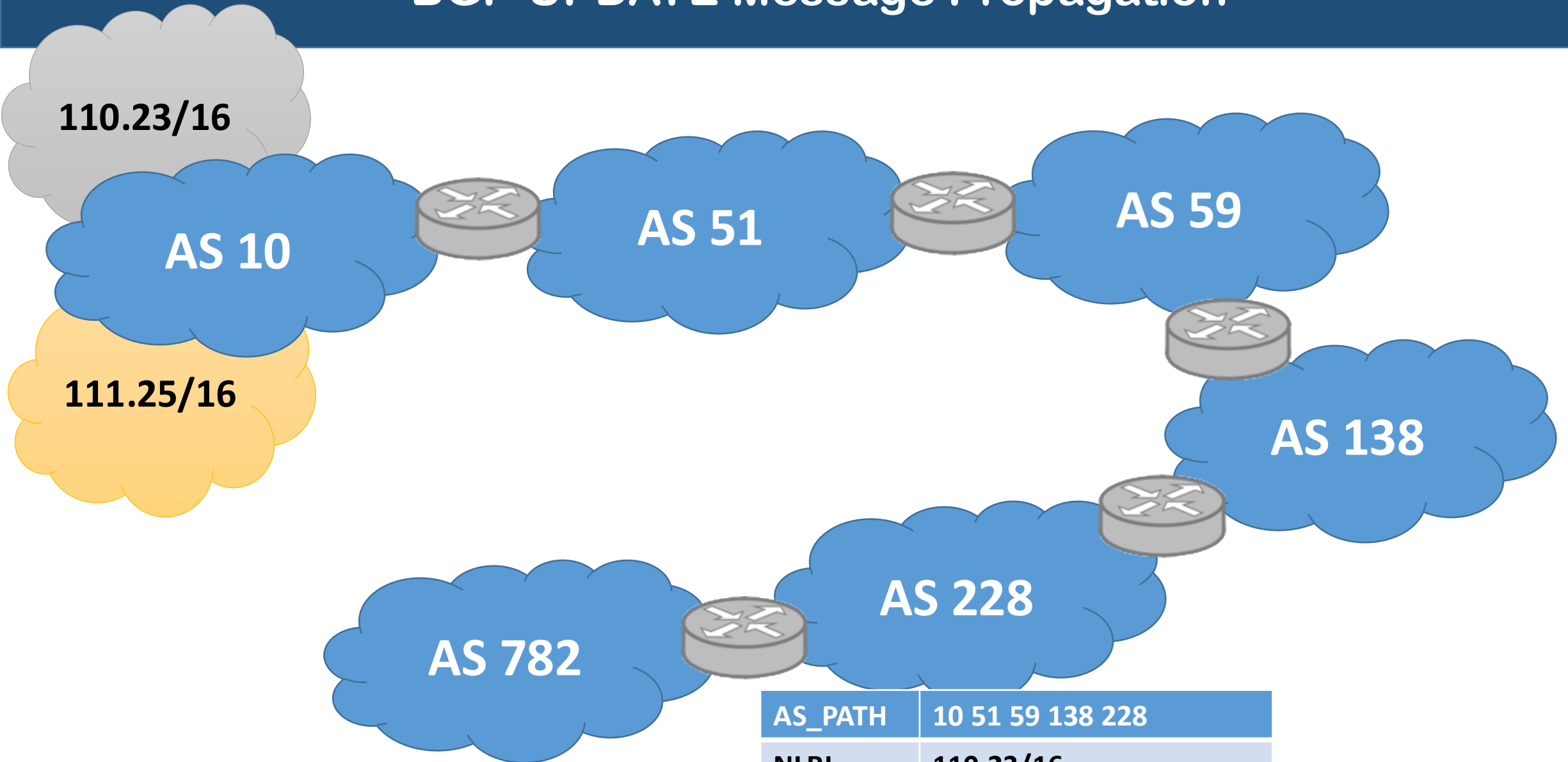
BGP UPDATE Message Propagation



BGP UPDATE Message Propagation

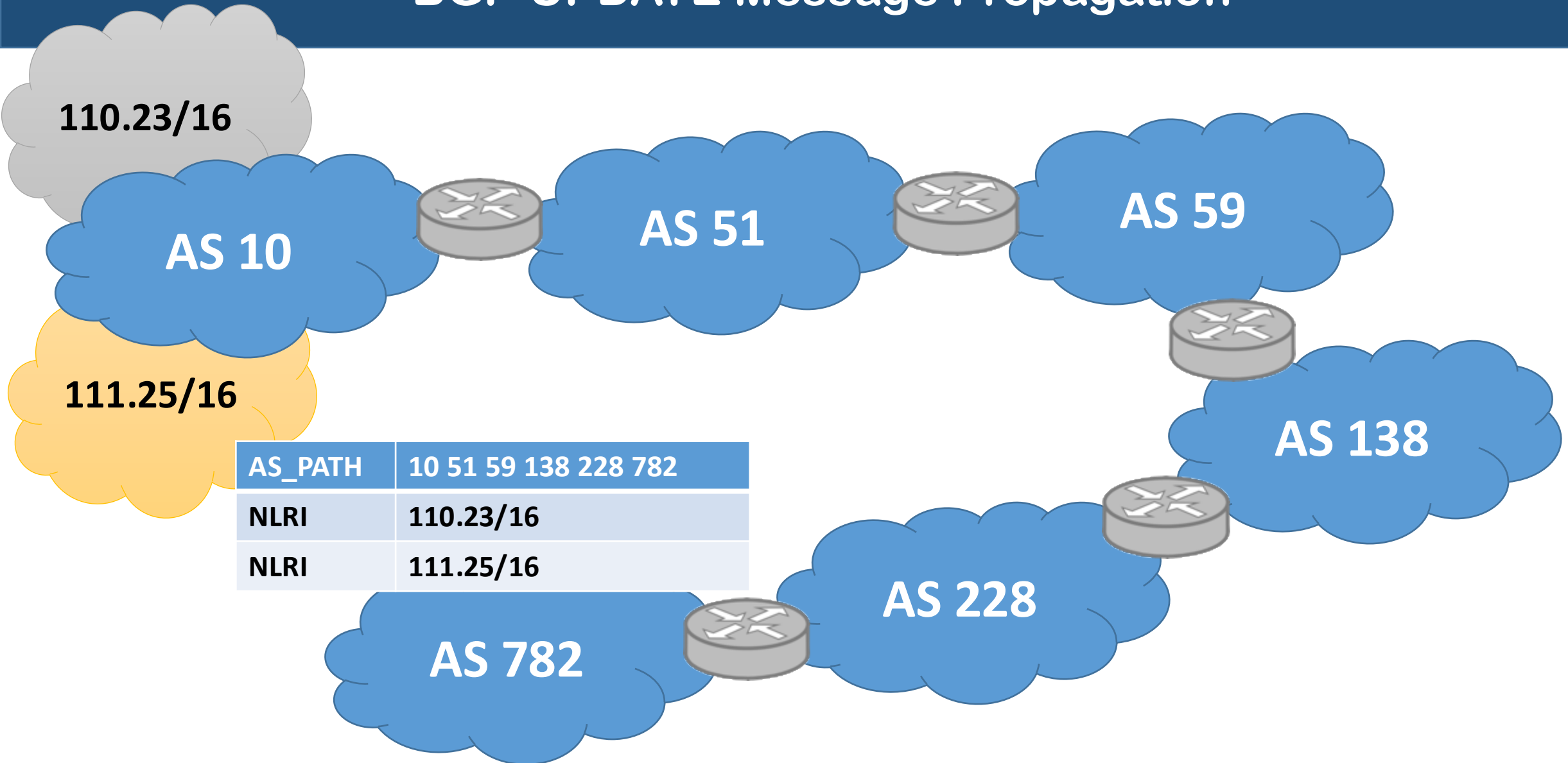


BGP UPDATE Message Propagation



AS_PATH	10 51 59 138 228
NLRI	110.23/16
NLRI	111.25/16

BGP UPDATE Message Propagation



BGP Route Establishment

- Based on the BGP UPDATE message, a BGP speaker may have multiple paths to a subnet
- The BGP speaker chooses the best one according to its own local policies
-> It advertises this route in the next BGP UPDATE message
 - Check the set of rules that are followed for BGP path establishment algorithm:
<https://www.cisco.com/c/en/us/support/docs/ip/border-gateway-protocol-bgp/13753-25.html>

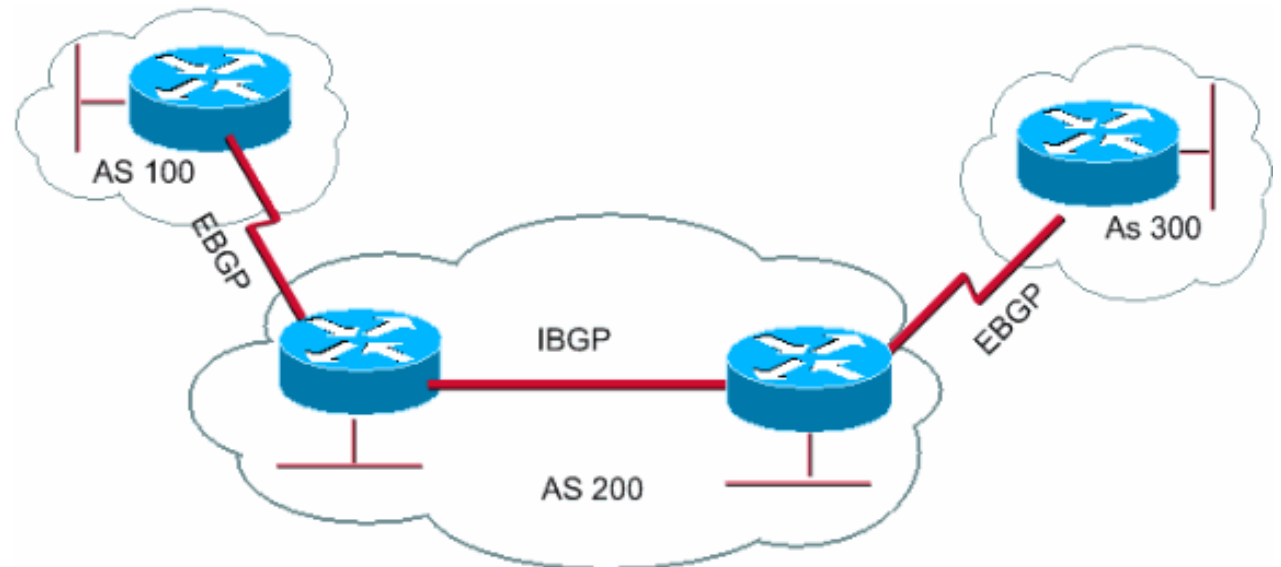


Image courtesy: <https://www.cisco.com>

BGP Looking Glass

BGP Looking Glass (203.110.242.19)

Reload this widget by entering a resource here

⚙️ Advanced Settings

- ▼ 2 RRCs see 19 peers announcing 203.110.242.19 originated by AS55847. [EXPAND EVERYTHING]
- ▼ RRC03 in Amsterdam, Netherlands sees 1 ASN originating 203.110.242.0/24.
(AS55847)
- ▼ AS55847 is seen as the origin by 8 peers.
 - ▶ 80.249.209.167 is announcing route AS6453 AS4755 AS9885 AS55824 AS55847.
 - ▶ 80.249.208.34 is announcing route AS1103 AS20965 AS9885 AS55824 AS55847.
 - ▶ 80.249.210.99 is announcing route AS50763 AS8943 AS2914 AS15412 AS18101 AS9885 AS55824 AS55847.
 - ▶ 80.249.211.217 is announcing route AS8455 AS3257 AS6453 AS4755 AS9885 AS55824 AS55847.

<https://stat.ripe.net/widget/looking-glass>